

claude-windows / e83b09d8-e113-4d4d-9411-cd80ae217af9

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ae83b09d8-e113-4d4d-9411-cd80ae217af9\subagents\workflows\wf_ae339437-da9\agent-ae8375cd5cc4e3892.jsonl`  
- SHA-256: `ab1f013fb12e334c95e3bbcb6431d769b8425770ce6eb5f7544422e9d95a57f2`  
- Source modified: `2026-07-07T15:54:09+00:00`  
- Imported at: `2026-07-08T15:59:35+00:00`  
- project: `wf_ae339437-da9`  
- session_id: `e83b09d8-e113-4d4d-9411-cd80ae217af9`
```

Transcript

1. user (2026-07-07T15:48:13.155Z)

You are reviewing ONE commit on branch `fix/cloud-run-exec-status-fastfail` in the repo at:

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer

Always start by: `cd "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer"` (the Bash cwd may default elsewhere). Use Read/Grep with absolute paths under that repo.

The commit hardens `CloudRunCompute.run\_infer` (backend/compute/cloud_run.py): the done-marker stays the
SOLE success signal, but each bucket-poll ALSO probes the Cloud Run Executions API so a worker that
TERMINATED without ever writing a marker (crash/OOM/an old image argparse-aborting on an unknown flag)
fails fast (raise RemoteExecutionFailed) instead of polling the full remote_poll_max_wait_sec (~65 min).
`\_dispatch\_remote` maps RemoteExecutionFailed -> rc 5 (distinct from the rc 4 poll-timeout).

CRITICAL CONTEXT: this commit was REBASED onto a master that had since merged two sibling PRs touching
the same files:
- #513 (23c1868): added `ComputeJobSpec.skip\_validation` + a `--skip-validation` worker flag;
the

control-plane dispatcher sets it so the worker skips redundant re-validation.
- #512 (20862d7): a validators shared-decode refactor (file-disjoint from this commit).
The motivating scenario for THIS fix: a post-#513 control-plane dispatching `--skip-validation`
to an
OLD worker image (no such flag) -> the worker argparse-aborts (exit 2) BEFORE writing a marker
-> the
exact ~65-min hang this fix collapses to a prompt rc-5 failure.

To see EXACTLY what this commit changes (only 5 files), run:

```
git diff origin/master...HEAD (three-dot: my change vs merge-base)
```

To see the sibling context that landed on the same files, run:

```
git diff 3be0935..origin/master -- backend/compute/base.py backend/compute/cloud_run.py  
backend/worker/__main__.py
```

Read the FULL current files (not just the diff) to judge correctness in context:

```
backend/compute/cloud_run.py, backend/compute/base.py, backend/compute/__init__.py,  
backend/worker/__main__.py, backend/orchestration.py (the API caller),  
tests/test_compute_backend.py,  
backend/infrastructure/execution_policy.py (poll_until semantics).
```

Report ONLY real, defect-grade findings with a concrete failure scenario. Do NOT report style nits,
things the gate already covers, or speculation. If the change is correct on your lens, return an

empty
findings array and say so in `assessment`. Full suite (1833) + ruff + mypy + coverage 85.66%
already pass;
do not re-report "add a test" unless a SPECIFIC untested behavior would plausibly break.

LENS ? REBASE/INTEGRATION CORRECTNESS. Did the rebase across #513 (skip_validation) and #512 (validators)
preserve BOTH sibling features AND this fix, with no silent drop, contradiction, dead code, or duplicated
logic? Specifically verify: (a) ComputeJobSpec still carries skip_validation AND run_infer still appends
--skip-validation; (b) _dispatch_remote builds the spec with skip_validation AND catches RemoteExecutionFailed
(rc 5) alongside PolicyTimeout (rc 4); (c) nothing this commit added shadows/breaks #513's behavior. Confirm the
two features are truly orthogonal or flag any real interaction.

2. assistant (2026-07-07T15:48:15.458Z)

I'll start by examining the repository state and the commit under review.

3. user (2026-07-07T15:48:18.158Z)

```
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
---BRANCH---
fix/cloud-run-exec-status-fastfail
---DIFF STAT---
 backend/compute/__init__.py | 8 +-
 backend/compute/base.py      | 14 +++
 backend/compute/cloud_run.py | 115 ++++++
 backend/worker/__main__.py   | 16 +-
 tests/test_compute_backend.py | 229 ++++++
5 files changed, 375 insertions(+), 7 deletions(-)
```

4. assistant (2026-07-07T15:48:20.976Z)

Let me look at the actual commit diff and the sibling context.

5. user (2026-07-07T15:48:23.653Z)

```
diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
index 1091582..482f3c9 100644
--- a/backend/compute/__init__.py
+++ b/backend/compute/__init__.py
@@ -13,13 +13,19 @@ import dataclasses
 import os
 from typing import Any, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,
+    RemoteExecutionFailed,
+)
 from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

__all__ = [
    "ComputeBackend",
```

```

    "ComputeJobSpec",
    "ComputeResult",
+   "RemoteExecutionFailed",
    "get_compute_backend",
]

diff --git a/backend/compute/base.py b/backend/compute/base.py
index 31af3fe..3ad76fc 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -17,6 +17,20 @@ from dataclasses import dataclass, field
    from typing import Callable, Optional

+class RemoteExecutionFailed(RuntimeError):
+    """A dispatched remote job's execution reached a TERMINAL state without ever writing its
+    completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image argparse-
+    rejecting
+    an unknown flag) before the marker push, or the marker upload itself failed on an otherwise
+    Succeeded run.
+
+    Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL be
+    running): this says the execution is DEFINITELY over and no marker will ever appear, so
+    the
+    dispatcher must stop bucket-polling immediately instead of burning the full
+    ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message carries the
+    job + execution name + terminal state + the ``gcloud`` command to read the execution log,
+    so
+    the whole "worker crashed before the done-marker" class fails fast AND diagnostically."""
+
+
+@dataclass(frozen=True)
+class ComputeJobSpec:
+    """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will (re)compute
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 7247ace..1e6e313 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -11,6 +11,10 @@ Flow (the cloud-serving compute path):
    job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-
    healthy
    worker that finished just past the deadline is rescued, not wasted ? then best-effort
    CANCELS
    the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
+    *Fast negative path:* while the marker stays the SOLE success signal, each poll also
+    probes the
+    Cloud Run Executions API ? if the execution has TERMINATED without ever writing a marker
+    (a
+    worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
+    diagnostically
+    (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile wait.
+    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
+    ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

@@ -32,7 +36,12 @@ import uuid
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,
+    RemoteExecutionFailed,
+)

```

```

from backend.infrastructure.execution_policy import (
    DEFAULT_POLICY,
    ExecutionPolicy,
@@ -76,6 +85,21 @@ class CloudRunJobClient(ABC):
    May raise ? the caller treats every failure as best-effort (the original poll timeout
is
    NEVER masked by a cancel error)."""

+ def execution_terminal_state(self, execution: str) -> Optional[str]:
+     """Return the execution's TERMINAL state as a short label (`"Failed"` /
+     `"Cancelled"` /
+     `"Succeeded"`) if it has finished, else `None` (still pending/running, or the state
+     can't be determined).
+
+     ADVISORY, not authoritative: it feeds `run_infer`'s fast negative path so a worker
that
+     terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails fast
instead
+     of bucket-polling the full budget. The done-marker stays the ONLY success signal.
+
+     NON-abstract with a default of `None` (additive / default-OFF): a client that can't ?
or
+     chooses not to ? poll the Executions API degrades transparently to today's marker-only
+     polling. Implementations MAY raise on a transient API error ? `run_infer` treats a
raise
+     as `None` (unknown ? keep waiting), so a flaky status API never fails a healthy
run."""
+     return None
+

class GoogleCloudRunJobClient(CloudRunJobClient):
    """Real client ? lazy-imports `google-cloud-run`. Owner-verified on the M7 real run; CI
uses
@@ -145,6 +169,35 @@ class GoogleCloudRunJobClient(CloudRunJobClient):
    ) # pragma: no cover - exercised only on a real run
    client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))

+ def execution_terminal_state(self, execution: str) -> Optional[str]:
+     # Advisory status probe (see the ABC docstring): describe the execution and, ONLY once
it
+     # is terminal (`completion_time` set), classify it. A non-resolvable name / absent
SDK
+     # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
+     # raises on bad input, because the probe is best-effort and must not fail a healthy
run.
+     if "/executions/" not in (execution or ""):
+         return None
+     try:
+         from google.cloud import run_v2 # type: ignore[attr-defined]
+     except Exception: # pragma: no cover - real SDK not present in CI
+         return None
+     client = (
+         run_v2.ExecutionsClient()
+     ) # pragma: no cover - exercised only on a real run
+     exe = client.get_execution( # pragma: no cover - real run only
+         request=run_v2.GetExecutionRequest(name=execution)
+     )
+     # completion_time is a proto Timestamp: unset ? seconds==0 ? still running/pending.
+     if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
+         return None # pragma: no cover - real run only
+     # Terminal: prefer the explicit per-task counts (a cancel shows as cancelled_count>0).
+     if getattr(exe, "cancelled_count", 0): # pragma: no cover - real run only
+         return "Cancelled"
+     if getattr(exe, "failed_count", 0): # pragma: no cover - real run only
+         return "Failed"

```

```

+         if getattr(exe, "succeeded_count", 0): # pragma: no cover - real run only
+             return "Succeeded"
+         return "Failed" # pragma: no cover - terminal but no positive success ? treat as
failed
+

class CloudRunCompute(ComputeBackend):
    """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
@@ -215,7 +268,7 @@ class CloudRunCompute(ComputeBackend):

    try:
        marker = poll_until(
-         lambda: self._read_marker(done_uri),
+         lambda: self._poll_check(done_uri, str(execution or "")),
        self._policy,
        label="cloud-run-infer",
        logger=LOG,
@@ -224,6 +277,9 @@ class CloudRunCompute(ComputeBackend):
        on_tick=on_wait,
    )
    except PolicyTimeout as timeout_exc:
+         # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
+         # so it propagates past here uncaught ? the dispatch handler maps it to a distinct,
+         # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
        marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
        video_id = str(marker.get("video_id", ""))
        # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
@@ -249,6 +305,61 @@ class CloudRunCompute(ComputeBackend):
        execution=str(execution or ""),
    )

+ def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
+     """One bucket-poll iteration, hardened with an Executions-API fast negative path.
+
+     The done-marker is the SOLE success signal and is checked FIRST ? a present marker
always
+     wins (even if the execution's status later flips), so a worker that wrote a SUCCESS *or
a
+     FAILURE* marker resolves through the normal path with its real returncode. Only when
the
+     marker is absent do we consult the execution status: a worker that TERMINATED without
ever
+     writing a marker (crash / OOM / an old image argparse-aborting on an unknown flag)
would
+     otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min) before
failing
+     opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
+     failure.
+
+     Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
+     ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over, no
marker
+     will ever appear)."""
+     marker = self._read_marker(done_uri)
+     if marker is not None:
+         return marker # success signal ? authoritative even if status later flips
+     state = self._terminal_execution_state(execution)
+     if state is None:
+         return None # still pending/running, or status unknown ? keep bucket-polling
+     # The execution is terminal but no marker is present. Re-check the marker ONCE to close
the

```

```

+         # race where the worker wrote it between our two reads (mirrors the post-timeout rescue
in
+         # _handle_poll_timeout). If it is genuinely absent, the process is gone and no marker
will
+         # EVER appear ? fail fast + diagnostically rather than wait out the whole poll budget.
+         marker = self._read_marker(done_uri)
+         if marker is not None:
+             return marker
+         raise RemoteExecutionFailed(
+             f"cloud-run: job={self._job_name} execution={execution or '<unknown>'} reached "
+             f"terminal state {state!r} with NO completion marker ? the worker exited before "
+             "writing one (crash / OOM / pre-marker abort, or a failed marker upload on an "
+             "otherwise-Succeeded run). The execution is over, so the marker will never appear;
"
+             "inspect the Cloud Run Job execution log to diagnose: "
+             f"`gccloud run jobs executions describe {execution or '<execution>'} "
+             f"--region {self._region}`."
+         )
+
+ def _terminal_execution_state(self, execution: str) -> Optional[str]:
+     """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A raising
or
+     flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
+     so any exception ? and an empty/unresolvable execution name ? degrades to ``None``
(unknown
+     ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
+     if not execution:
+         return None
+     try:
+         return self._client.execution_terminal_state(execution)
+     except Exception: # noqa: BLE001 ? advisory probe: never let it kill the bucket-poll
+         LOG.debug(
+             "cloud-run: execution status probe failed for %s ? treating as unknown",
+             execution,
+             exc_info=True,
+         )
+         return None
+
+ def _handle_poll_timeout(
+     self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
+ ) -> dict:
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 9b19215..6d3420b 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -176,9 +176,14 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
    container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.

- A remote job that never writes a completion marker (a failed/hung container) makes the
bucket
- poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote
dispatch
- timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
+ A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
+ from the local 2/3), never a raw traceback:
+ * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution may
still be
+ running (the shim best-effort cancels it).
+ * rc 5 ? the execution has TERMINATED without a marker (``RemoteExecutionFailed``): a

```

```

worker
+         crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead of the
+         ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
+     from backend.compute import RemoteExecutionFailed
+     from backend.infrastructure.execution_policy import PolicyTimeout

    spec = ComputeJobSpec(
@@ -190,6 +195,11 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    )
    try:
        result = compute.run_infer(spec)
+     except RemoteExecutionFailed as e:
+         logger.error(
+             "[worker] remote execution terminated without a completion marker: %s", e
+         )
+     return 5
    except PolicyTimeout as e:
        logger.warning(
            "[worker] remote compute did not complete (no marker within budget): %s", e
diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py
index faa4981..07e8068 100644
--- a/tests/test_compute_backend.py
+++ b/tests/test_compute_backend.py
@@ -12,7 +12,11 @@ import tempfile

import pytest

-from backend.compute import ComputeJobSpec, get_compute_backend
+from backend.compute import (
+    ComputeJobSpec,
+    RemoteExecutionFailed,
+    get_compute_backend,
+)
+from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
+from backend.config import AppConfig
+from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout
@@ -110,6 +114,37 @@ class _NeverDoneClient(CloudRunJobClient):
    raise self._cancel_error

+class _CrashedNoMarkerClient(CloudRunJobClient):
+    """Simulates the bug's failure class: the worker execution TERMINATES (crash / OOM / an old
+    image argparse-aborting on an unknown flag) WITHOUT ever writing a done-marker. Dispatch
+    returns
+    a resolvable execution name, no marker is ever written, and the Executions API reports a
+    terminal
+    state ? so run_infer must fail FAST via the status probe instead of spinning the poll
+    budget."""
+    EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-crash"
+
+    def __init__(self, *, state: str = "Failed"):
+        self.calls: list[dict] = []
+        self.cancelled: list[str] = []
+        self.status_polls = 0
+        self._state = state
+
+    def run_job(self, job_name, argv, *, region, project=None):
+        self.calls.append(
+            {"job": job_name, "argv": list(argv), "region": region, "project": project}
+        )
+        return self.EXEC_NAME
+
+    def cancel_execution(self, execution):
+        self.cancelled.append(execution)

```

```

+
+ def execution_terminal_state(self, execution):
+     self.status_polls += 1
+     # Anti-hang guard: the fast path MUST abort on the first terminal observation. If a
+     # regression kept polling despite a terminal status, trip here instead of spinning.
+     assert self.status_polls <= 5, "status probed repeatedly ? fast path did not abort the
poll"
+     return self._state
+
+
+ # ----- factory (default-OFF)

@@ -423,6 +458,198 @@ def test_poll_timeout_late_marker_rescues_result(monkeypatch):
    assert exists_calls["n"] == 2 # exactly ONE extra post-timeout check

+
+
+## ----- execution-status fast negative path (worker crashed pre-
+marker)
+
+
+## poll_every_n_sec=0.0 ? the poll spins as fast as possible; max_wait_sec=2.0 BOUNDS any
+regression
+## (a fast path that silently fell back to marker-only polling fails in ~2s here, never a 65-min
+hang)
+## while the working fast path aborts on poll 0 ? instantly.
+_FAILFAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=2.0)
+
+
+def test_run_infer_fails_fast_when_execution_terminal_without_marker(tmp_path):
+    """The core fix: a worker execution that TERMINATED (Failed) without ever writing a marker
+is
+caught by the Executions-API probe ? RemoteExecutionFailed FAST (poll 0), not a ~65-min
+opaque
+hang. The diagnostic names the execution, the terminal state, and the log-inspection
+command."""
+    client = _CrashedNoMarkerClient(state="Failed")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )
+    with pytest.raises(RemoteExecutionFailed) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    msg = str(exc.value)
+    assert _CrashedNoMarkerClient.EXEC_NAME in msg # operator can find the execution
+    assert "Failed" in msg # the terminal state is surfaced
+    assert "gcloud run jobs executions describe" in msg # actionable log pointer
+    assert client.status_polls == 1 # aborted on the FIRST terminal observation (fast, not
+spun)
+    assert client.cancelled == [] # a finished execution is never cancelled (nothing to
+cancel)
+
+
+def test_dispatch_remote_maps_terminal_without_marker_to_rc5(tmp_path):
+    """The dispatch handler maps RemoteExecutionFailed to a distinct, clearly-labelled rc 5 ?
+NOT
+the rc 4 poll-timeout path (a still-maybe-running execution) and NOT a raw traceback ? so
+'worker crashed before the marker' is operationally distinguishable and surfaces
+immediately."""
+    import argparse
+
+

```



```

+ from backend.worker import __main__ as wmain
+
+ client = _CrashedNoMarkerClient(state="Failed")
+ backend = CloudRunCompute(
+     run_client=client,
+     job_name="rally-worker",
+     region="asia-south1",
+     policy=_FAILFAST,
+     token="tok",
+ )
+ args = argparse.Namespace(
+     video_uri="gs://b/v.mp4",
+     out_uri=str(tmp_path / "bucket"),
+     segmenter="wasb_hybrid",
+     set=None,
+ )
+ rc = wmain._dispatch_remote(backend, args)
+ assert rc == 5 # the crashed-pre-marker fast-fail code...
+ assert rc != 4 # ...distinct from the poll-budget timeout
+ assert client.status_polls == 1 # reached the fast path (not the timeout branch)
+
+
+def test_succeeded_state_without_marker_also_fails_fast(tmp_path):
+    """Generalization: ANY terminal state with no marker is a guaranteed-futile wait. A
+    'Succeeded'
+    execution that nonetheless wrote no marker (the marker upload itself failed) also fails
+    fast,
+    with the state surfaced ? the whole 'execution over, no marker' class collapses, not just
+    Failed."""
+    client = _CrashedNoMarkerClient(state="Succeeded")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )
+    with pytest.raises(RemoteExecutionFailed) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert "Succeeded" in str(exc.value)
+
+
+def test_present_marker_wins_over_terminal_status(tmp_path):
+    """The done-marker stays AUTHORITATIVE: a worker that wrote a marker ? even a FAILURE
+    marker
+    (rc 2) ? resolves through the normal path with its REAL returncode, never a synthesized
+    RemoteExecutionFailed, even when the execution status also reads terminal. Status is a
+    negative
+    path only; it must never override a real marker (which is checked first)."""
+
+    class _MarkerAndFailedStatus(_FakeRunClient):
+        def execution_terminal_state(self, execution): # pragma: no cover - never reached
+            return "Failed" # the marker resolves poll 0, so this is never consulted
+
+    client = _MarkerAndFailedStatus(video_id="abc", returncode=2)
+    backend = CloudRunCompute(
+        run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
+    )
+    result = backend.run_infer(
+        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+    )
+    assert result.returncode == 2 # the worker's REAL rc, not RemoteExecutionFailed
+    assert result.video_id == "abc"

```

```

+
+
+def test_status_probe_error_is_swallowed_and_falls_back_to_marker_polling(tmp_path):
+    """Best-effort probe: a raising Executions API must NEVER fail a healthy run. The wait
degrades
+    to today's marker-only polling ? PolicyTimeout (the existing timeout path), NOT
+    RemoteExecutionFailed ? a flaky status API cannot manufacture a failure."""
+
+    class _RaisingStatusClient(_NeverDoneClient):
+        def execution_terminal_state(self, execution):
+            raise RuntimeError("executions API unreachable")
+
+    client = _RaisingStatusClient()
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_TIMEOUT_NOW,
+        token="tok",
+    )
+    with pytest.raises(PolicyTimeout): # swallowed probe error ? the unchanged timeout path
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert client.cancelled == [_NeverDoneClient.EXEC_NAME] # still cancels on the real
timeout
+
+
+def test_default_execution_terminal_state_is_none():
+    """The ABC default is None (additive / default-OFF): a client that doesn't override status
+    polling degrades transparently to marker-only behaviour, so every pre-existing fake is
+    unaffected and the seam never changes a healthy run's outcome."""
+    assert _NoopRunClient().execution_terminal_state("whatever") is None
+    assert (
+        _NeverDoneClient().execution_terminal_state(_NeverDoneClient.EXEC_NAME) is None
+    )
+
+
+def test_real_client_terminal_state_none_for_unresolvable_name():
+    """The real client's status probe is best-effort: a non-resolvable execution name returns
None
+    (can't probe ? keep marker-polling), NOT a raise ? exercisable in CI (the SDK is
absent)."""
+    from backend.compute.cloud_run import GoogleCloudRunJobClient
+
+    client = GoogleCloudRunJobClient()
+    for bogus in ("", "execution", "exec-123"):
+        assert client.execution_terminal_state(bogus) is None
+
+
+def test_terminal_status_with_marker_written_in_race_resolves_via_marker(monkeypatch):
+    """Concurrency race: the execution reads terminal exactly as the worker finishes writing
its
+    marker. _poll_check re-checks the marker ONCE after a terminal status ? if it landed in
between,
+    the completed result is ingested (the marker wins), NOT a spurious
RemoteExecutionFailed."""
+    from backend.compute import cloud_run as cr
+
+    exists_calls = {"n": 0}
+
+    class _FakeStore:
+        def exists(self, ref):
+            exists_calls["n"] += 1
+            # absent on the pre-status read, present on the ONE post-status race re-check

```

```

+         return exists_calls["n"] >= 2
+
+ monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
+
+ def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+     if dst is None:
+         dst = os.path.join(tempfile.mkdtemp(prefix="cr-race-"), "m.json")
+         with open(dst, "w", encoding="utf-8") as f:
+             f.write({'"video_id": "vrace", "returncode": 0, "segment_count": 1}')
+         return dst
+
+ monkeypatch.setattr(cr, "fetch", fake_fetch)
+ client = _CrashedNoMarkerClient(state="Failed") # status says terminal-Failed...
+ backend = cr.CloudRunCompute(
+     run_client=client, job_name="j", region="r", token="t", policy=_FAILFAST
+ )
+ result = backend.run_infer(
+     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
+ )
+ assert result.video_id == "vrace" # ...but the marker that raced in WINS
+ assert exists_calls["n"] == 2 # exactly the pre-status read + the ONE race re-check
+ assert client.status_polls == 1 # status was consulted once, then the marker superseded it
+
+
+def test_terminal_execution_state_none_for_empty_execution():
+    """An empty/unknown execution name (run_job could not resolve one) can't be probed ? None
+    (degrade to marker-only polling), and the client's status API is never even called."""
+
+    class _Boom(_NoopRunClient):
+        def execution_terminal_state(self, execution): # pragma: no cover - must never be
called
+            raise AssertionError("must not probe an empty execution name")
+
+    backend = CloudRunCompute(run_client=_Boom(), job_name="j", region="r")
+    assert backend._terminal_execution_state("") is None
+
+
+    # ----- worker wiring: dispatch branch + done-
marker

```

6. assistant (2026-07-07T15:48:26.091Z)

Now let me read the full current files to judge correctness in context.

7. user (2026-07-07T15:48:27.683Z)

```

1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** `done_uri` via `execution_policy.poll_until` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll. On a poll TIMEOUT (`max_wait_sec` can
undercut the
11        job's own `--task-timeout` budget) the shim does ONE final marker check ? a slow-
but-healthy

```

```

12         worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13         the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14         *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15         Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16         worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17         (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18         3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19         ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21         The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22         so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24         The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
25         with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26         owner-verified on the M7 real run.
27         ""
28
29         from __future__ import annotations
30
31         import json
32         import logging
33         import os
34         import tempfile
35         import uuid
36         from abc import ABC, abstractmethod
37         from typing import Any, Callable, List, Optional
38
39         from backend.compute.base import (
40             ComputeBackend,
41             ComputeJobSpec,
42             ComputeResult,
43             RemoteExecutionFailed,
44         )
45         from backend.infrastructure.execution_policy import (
46             DEFAULT_POLICY,
47             ExecutionPolicy,
48             PolicyTimeout,
49             poll_until,
50         )
51         from backend.storage import fetch, get_storage, parse_uri
52
53         LOG = logging.getLogger("compute.cloud_run")
54
55         # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
56         # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
57         _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
58             "deployment.compute_target=local_gpu",
59             "indexing.detector_impl=native",
60         )
61
62
63         class CloudRunJobClient(ABC):
64             """Triggers a Cloud Run **Job** execution with per-execution container arg

```

```

overrides.
65
66         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
67         overrides the job's container ``args`` for this execution and starts it."""
68
69         @abstractmethod
70         def run_job(
71             self,
72             job_name: str,
73             argv: List[str],
74             *,
75             region: str,
76             project: Optional[str] = None,
77         ) -> str:
78             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
79             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
80
81         @abstractmethod
82         def cancel_execution(self, execution: str) -> None:
83             """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
84             times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
85             May raise ? the caller treats every failure as best-effort (the original poll
timeout is
86             NEVER masked by a cancel error)."""
87
88         def execution_terminal_state(self, execution: str) -> Optional[str]:
89             """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
90             ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
91             can't be determined).
92
93             ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
94             terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95             of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97             NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98             chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99             polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100             as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101             return None
102
103
104         class GoogleCloudRunJobClient(CloudRunJobClient):
105             """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106             a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108             def run_job(
109                 self,
110                 job_name: str,
111                 argv: List[str],

```

```

112         *,
113         region: str,
114         project: Optional[str] = None,
115     ) -> str:
116         # A None/empty project would build a bogus "projects/None/locations/..." path
117         (job_path
118         # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
119         exercisable
120         # in CI (where google-cloud-run is absent) and so mypy narrows project to str
121         for job_path.
122         if not project:
123             raise RuntimeError(
124                 "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
125                 project id "
126                 "to resolve the job path; set it on the control plane (see
127                 get_compute_backend)."
128             )
129         try:
130             from google.cloud import run_v2 # type: ignore[attr-defined]
131         except Exception as exc: # pragma: no cover - real SDK not present in CI
132             raise RuntimeError(
133                 "google-cloud-run is required for CloudRunCompute's real client; install
134                 it on the "
135                 "control plane (it is intentionally NOT a CI/test dependency)."
136             ) from exc
137         client = (
138             run_v2.JobsClient()
139         ) # pragma: no cover - exercised only on the real M7 run
140         name = client.job_path(project, region, job_name)
141         overrides = run_v2.RunJobRequest.Overrides(
142             container_overrides=[
143                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
144             ]
145         )
146         op = client.run_job(
147             request=run_v2.RunJobRequest(name=name, overrides=overrides)
148         )
149         return (
150             getattr(op, "metadata", None)
151             and getattr(op.metadata, "name", "")
152             or "execution"
153         )
154
155     def cancel_execution(self, execution: str) -> None:
156         # run_job falls back to the literal string "execution" when the operation
157         metadata carries
158         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
159         import
160         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
161         (where
162         # google-cloud-run is absent).
163         if "/executions/" not in (execution or ""):
164             raise RuntimeError(
165                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
166                 expected "
167                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
168                 resolve "
169                 "one from the operation metadata, so there is nothing to cancel by
170                 name)."
171             )
172         try:
173             from google.cloud import run_v2 # type: ignore[attr-defined]
174         except Exception as exc: # pragma: no cover - real SDK not present in CI
175             raise RuntimeError(
176                 "google-cloud-run is required to cancel a Cloud Run execution; install

```

```

it on the "
165         "control plane (it is intentionally NOT a CI/test dependency)."
166     ) from exc
167     client = (
168         run_v2.ExecutionsClient()
169     ) # pragma: no cover - exercised only on a real run
170     client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
171
172     def execution_terminal_state(self, execution: str) -> Optional[str]:
173         # Advisory status probe (see the ABC docstring): describe the execution and,
ONLY once it
174         # is terminal (`completion_time` set), classify it. A non-resolvable name /
absent SDK
175         # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this
never
176         # raises on bad input, because the probe is best-effort and must not fail a
healthy run.
177         if "/executions/" not in (execution or ""):
178             return None
179         try:
180             from google.cloud import run_v2 # type: ignore[attr-defined]
181         except Exception: # pragma: no cover - real SDK not present in CI
182             return None
183         client = (
184             run_v2.ExecutionsClient()
185         ) # pragma: no cover - exercised only on a real run
186         exe = client.get_execution( # pragma: no cover - real run only
187             request=run_v2.GetExecutionRequest(name=execution)
188         )
189         # completion_time is a proto Timestamp: unset ? seconds==0 ? still
running/pending.
190         if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
191             return None # pragma: no cover - real run only
192         # Terminal: prefer the explicit per-task counts (a cancel shows as
cancelled_count>0).
193         if getattr(exe, "cancelled_count", 0): # pragma: no cover - real run only
194             return "Cancelled"
195         if getattr(exe, "failed_count", 0): # pragma: no cover - real run only
196             return "Failed"
197         if getattr(exe, "succeeded_count", 0): # pragma: no cover - real run only
198             return "Succeeded"
199         return "Failed" # pragma: no cover - terminal but no positive success ? treat
as failed
200
201
202     class CloudRunCompute(ComputeBackend):
203         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
204
205         def __init__(
206             self,
207             *,
208             run_client: CloudRunJobClient,
209             job_name: str,
210             region: str,
211             project: Optional[str] = None,
212             storage_config: Optional[dict] = None,
213             policy: ExecutionPolicy = DEFAULT_POLICY,
214             token: Optional[str] = None,
215             container_overrides: Optional[tuple[str, ...]] = None,
216         ) -> None:
217             self._client = run_client
218             self._job_name = job_name
219             self._region = region
220             self._project = project
221             self._storage_config = storage_config or {}

```

```

222         self._policy = policy
223         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
224         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
225         self._token = token
226         self._container_overrides = (
227             _DEFAULT_CONTAINER_OVERRIDES
228             if container_overrides is None
229             else tuple(container_overrides)
230         )
231
232     def run_infer(
233         self,
234         spec: ComputeJobSpec,
235         *,
236         on_wait: "Callable[[float], None] | None" = None,
237     ) -> ComputeResult:
238         out_root = spec.out_uri.rstrip("/")
239         token = self._token or uuid.uuid4().hex
240         done_uri = f"{out_root}/_dispatch/{token}.done"
241         argv: List[str] = [
242             "infer",
243             "--video-uri",
244             spec.video_uri,
245             "--out-uri",
246             spec.out_uri,
247             "--done-uri",
248             done_uri,
249         ]
250         if spec.segmenter:
251             argv += ["--segmenter", spec.segmenter]
252         if spec.skip_validation:
253             # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
254             # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
255             argv.append("--skip-validation")
256         for kv in (*spec.config_overrides, *self._container_overrides):
257             argv += ["--set", kv]
258
259         execution = self._client.run_job(
260             self._job_name, argv, region=self._region, project=self._project
261         )
262         LOG.info(
263             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
264             self._job_name,
265             execution,
266             done_uri,
267         )
268
269         try:
270             marker = poll_until(
271                 lambda: self._poll_check(done_uri, str(execution or "")),
272                 self._policy,
273                 label="cloud-run-infer",
274                 logger=LOG,
275                 # Live heartbeat (#482): each still-waiting tick reaches the caller so
276                 # job.progress moves during the whole GPU run instead of freezing.
277                 on_tick=on_wait,
278             )
279         except PolicyTimeout as timeout_exc:
280             # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
281             # so it propagates past here uncaught ? the dispatch handler maps it to a

```



```

distinct,
282         # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
283         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
284         video_id = str(marker.get("video_id", ""))
285         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
286         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
287         rc = int(marker.get("returncode", 1))
288         if not video_id:
289             LOG.warning(
290                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
291                 "video_id) ? treating as failure",
292                 self._job_name,
293             )
294             rc = rc or 1
295         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
296         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
297         LOG.info(
298             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
299         )
300         return ComputeResult(
301             returncode=rc,
302             outputs_uri=outputs_uri,
303             video_id=video_id,
304             report=report,
305             execution=str(execution or ""),
306         )
307
308     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
309         """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
310
311         The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
312         wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
313         FAILURE* marker resolves through the normal path with its real returncode. Only
when the
314         marker is absent do we consult the execution status: a worker that TERMINATED
without ever
315         writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
316         otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
317         opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
318         failure.
319
320         Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
321         ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
322         will ever appear)."""
323         marker = self._read_marker(done_uri)
324         if marker is not None:
325             return marker # success signal ? authoritative even if status later flips
326             state = self._terminal_execution_state(execution)
327             if state is None:
328                 return None # still pending/running, or status unknown ? keep bucket-
polling
329             # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
330             # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in

```

```

331         # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
332         # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
333         marker = self._read_marker(done_uri)
334         if marker is not None:
335             return marker
336         raise RemoteExecutionFailed(
337             f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
338             f"terminal state {state!r} with NO completion marker ? the worker exited
before "
339             "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
340             "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
341             "inspect the Cloud Run Job execution log to diagnose: "
342             f"`gcloud run jobs executions describe {execution or '<execution>'} "
343             f"--region {self._region}`."
344         )
345
346     def _terminal_execution_state(self, execution: str) -> Optional[str]:
347         """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
348         flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
349         so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
350         ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
351         if not execution:
352             return None
353         try:
354             return self._client.execution_terminal_state(execution)
355         except Exception: # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
356             LOG.debug(
357                 "cloud-run: execution status probe failed for %s ? treating as unknown",
358                 execution,
359                 exc_info=True,
360             )
361             return None
362
363     def _handle_poll_timeout(
364         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
365     ) -> dict:
366         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
367
368         The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
369         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
370         ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
371         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
372         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
373         execution name so the operator can verify the state in the GCP console. A cancel
failure
374         NEVER masks the original timeout."""
375         try:
376             late = self._read_marker(done_uri)
377         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
378             LOG.warning(

```

```

379         "cloud-run: final post-timeout marker check failed ? treating as
absent",
380         exc_info=True,
381     )
382     late = None
383     if late is not None:
384         LOG.warning(
385             "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
386             "? rescuing the completed result",
387             self._job_name,
388             execution,
389         )
390     return late
391     try:
392         self._client.cancel_execution(execution)
393     except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
394         cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
395         LOG.warning(
396             "cloud-run: best-effort cancel of execution=%s failed: %s",
397             execution,
398             cancel_exc,
399             exc_info=True,
400         )
401     else:
402         cancel_note = "cancel requested"
403         LOG.info("cloud-run: requested cancel of execution=%s", execution)
404     raise PolicyTimeout(
405         f"{timeout_exc} [cloud-run job={self._job_name} "
406         f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
407         "state in the GCP console]"
408     ) from timeout_exc
409
410     # --- storage-backed completion signal (bucket-poll) ---
411     def _read_marker(self, done_uri: str) -> Optional[dict]:
412         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
413         ref = parse_uri(done_uri)
414         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
415             return None
416         return (
417             self._read_json(done_uri) or {}
418         ) # empty dict still terminates the poll (present == done)
419
420     def _read_json(self, uri: str) -> dict:
421         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
422         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
423         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
424         try:
425             ref = parse_uri(uri)
426             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
427                 dst = os.path.join(td, ref.name or "obj.json")
428                 local = fetch(uri, dst, config=self._storage_config)
429                 with open(local, "r", encoding="utf-8") as f:
430                     data: Any = json.load(f)
431                 return data if isinstance(data, dict) else {}
432         except Exception:
433             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
434             return {}
435

```

8. user (2026-07-07T15:48:27.847Z)

```
1      """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3      COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4      (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5      core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6      (cloud-serving) ? without the core ever branching on the profile (D1).
7
8      The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9      non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Callable, Optional
18
19
20     class RemoteExecutionFailed(RuntimeError):
21         """A dispatched remote job's execution reached a TERMINAL state without ever writing
its
22         completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image
argparse-rejecting
23         an unknown flag) before the marker push, or the marker upload itself failed on an
otherwise
24         Succeeded run.
25
26         Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL
be
27         running): this says the execution is DEFINITELY over and no marker will ever
appear, so the
28         dispatcher must stop bucket-polling immediately instead of burning the full
29         ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message
carries the
30         job + execution name + terminal state + the ``gcloud`` command to read the execution
log, so
31         the whole "worker crashed before the done-marker" class fails fast AND
diagnostically."""
32
33
34     @dataclass(frozen=True)
35     class ComputeJobSpec:
36         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
37         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
38         deterministic regardless of where the job ran."""
39
40         video_uri: str # input (gs://? for cloud; a path/local:// for local)
41         out_uri: str # output ROOT (gs://? bucket for cloud)
42         segmenter: Optional[str] = None
43         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
44         config_overrides: tuple[str, ...] = ()
45         # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving
control-plane) is
46         # the authoritative validation gate: it runs the SAME validators with ITS resolved
config and only
47         # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video
```

```

decode, and
48         # (c) outright WRONG when the worker's config resolves different thresholds ? it can
REJECT a clip
49         # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no
control-plane)
50         # validating exactly as before.
51         skip_validation: bool = False
52
53
54     @dataclass(frozen=True)
55     class ComputeResult:
56         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
57         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
58         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
59         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
60
61         returncode: int
62         outputs_uri: str = ""
63         video_id: str = ""
64         report: dict = field(default_factory=dict)
65         # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
66         # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
67         # executions describe <execution>). Empty for the in-process path / backends without
one.
68         execution: str = ""
69
70
71     class ComputeBackend(ABC):
72         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
73         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
74         execution identically."""
75
76         @abstractmethod
77         def run_infer(
78             self,
79             spec: ComputeJobSpec,
80             *,
81             on_wait: "Callable[[float], None] | None" = None,
82         ) -> ComputeResult:
83             """Run ``spec`` to completion and return its ``ComputeResult``.
84
85             ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
86             still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
87             long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
88

```

9. assistant (2026-07-07T15:48:36.304Z)

Now let me look at the sibling context and the worker/orchestration files.

10. user (2026-07-07T15:48:39.021Z)

```

diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py

```

```

+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
    segmenter: Optional[str] = None
    # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
    frozen/hashable.
    config_overrides: tuple[str, ...] = ()
+   # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
+   plane) is
+   # the authoritative validation gate: it runs the SAME validators with ITS resolved config
+   and only
+   # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
+   and
+   # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
+   a clip
+   # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
+   plane)
+   # validating exactly as before.
+   skip_validation: bool = False

    @dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
    ]
    if spec.segmenter:
        argv += ["--segmenter", spec.segmenter]
+   if spec.skip_validation:
+       # The control-plane already validated + only dispatches on PASS ? tell the worker
+       not to
+       # re-decode and re-validate (redundant, and its config may resolve different
+       thresholds).
+       argv.append("--skip-validation")
    for kv in (*spec.config_overrides, *self._container_overrides):
        argv += ["--set", kv]

diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    out_uri=args.out_uri,
    segmenter=args.segmenter,
    config_overrides=tuple(args.set or ()),
+   skip_validation=bool(getattr(args, "skip_validation", False)),
    )
    try:
        result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
    identical).
    video_id = compute_video_id(local_video)

-   # 3. VALIDATE (same registry as the main CLI).
-   val_results, val_context = validator_registry.run_all_with_context(
-       local_video, config
-   )
+   # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+   # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
+   the
+   # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
+   is
+   # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's

```

```

config
+ # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+ # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+ # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+ # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+ # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+ if getattr(args, "skip_validation", False):
+     print(
+         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+         "gate and already validated this input."
+     )
+     val_results, val_context = [], {}
+ else:
+     val_results, val_context = validator_registry.run_all_with_context(
+         local_video, config
+     )
+     failures = [r for r in val_results if not r.passed]
+     db = Database(os.path.join(scratch, config.output.db_name))
+     db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
+         metavar="k=v",
+         help="config override, e.g. --set indexing.skip_ai_handoff=true",
+     )
+     pi.add_argument(
+         "--skip-validation",
+         action="store_true",
+         help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+         "validators with its resolved config and only dispatches on PASS, so re-validating here
is "
+         "redundant, decodes the video again, and can contradict the gate. Default-OFF: the
direct "
+         "``worker infer`` CLI still validates.",
+     )
+     pi.set_defaults(func=cmd_infer)

pt = sub.add_parser(

```

11. user (2026-07-07T15:48:39.405Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """
16
17    from __future__ import annotations
18

```

```

19     import argparse
20     import csv
21     import json
22     import logging
23     import os
24     import re
25     import sys
26     import tempfile
27     from typing import Any, List
28
29     from backend import storage
30     from backend.compute import ComputeJobSpec, get_compute_backend
31     from backend.config import (
32         StartupCheckError,
33         enforce_startup_checks,
34         load_config,
35         probe_cuda_available,
36     )
37     from backend.infrastructure.database import Database
38     from backend.pipeline.segmenters.base import segmenter_registry
39     from backend.pipeline.validators.base import registry as validator_registry
40     from backend.reporting import emit_indexer_report
41     from backend.results import Failure
42     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("backend.worker")
46
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50         str."""
51         low = v.lower()
52         if low in ("true", "false"):
53             return low == "true"
54         for cast in (int, float):
55             try:
56                 return cast(v)
57             except ValueError:
58                 pass
59         return v
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
63         indexing.skip_ai_handoff=true)."""
64         for kv in sets or []:
65             key, sep, val = kv.partition("=")
66             if not sep:
67                 raise ValueError(f"--set expects k=v, got {kv!r}")
68             parts = key.split(".")
69             obj = config
70             for p in parts[:-1]:
71                 obj = getattr(obj, p)
72             setattr(obj, parts[-1], _coerce(val))
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
76         friendly
77         artifact (same schema as the rally-annotator labels)."""
78         with open(path, "w", newline="") as f:
79             w = csv.writer(f)
80             w.writerow(["start", "end", "shot_count", "ending_reason"])
81             for s in segments:

```



```

81         w.writerow(
82             [
83                 f"{float(s.get('start_time', 0.0)):.3f}",
84                 f"{float(s.get('end_time', 0.0)):.3f}",
85                 s.get("shot_count", ""),
86                 s.get("ending_reason", s.get("shot_count_confidence", "")),
87             ]
88         )
89
90
91 def _write_report_json(
92     video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93 ) -> None:
94     with open(path, "w") as f:
95         json.dump(
96             {
97                 "video_id": video_id,
98                 "status": status,
99                 "segment_count": len(segments),
100                 # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                 # control plane re-hydrating from outputs/<md5>/ restores the video row
102                 # with its truthful path ? without this, only the segments recover.
103                 "video_uri": video_uri,
104                 "segments": [
105                     {
106                         "start": float(s.get("start_time", 0.0)),
107                         "end": float(s.get("end_time", 0.0)),
108                     }
109                     for s in segments
110                 ],
111             },
112             f,
113             indent=2,
114         )
115
116
117 def _serialize_and_push_outputs(
118     scratch: str,
119     out_uri: str,
120     video_id: str,
121     segments: List[dict],
122     status: str,
123     storage_cfg: dict,
124     video_uri: str = "",
125 ) -> List[str]:
126     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
127     push both to
128     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
129
130     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
131     timeout/abort leaves a
132     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
133     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
134     non-``success``
135     report as "no usable result" ? retry, and the restart-recovery poll waits for
136     report.json to
137     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
138     """
139     out_local = os.path.join(scratch, "outputs", video_id)
140     os.makedirs(out_local, exist_ok=True)
141     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
142     _write_report_json(
143         video_id,
144         segments,
145         status,
146         os.path.join(out_local, "report.json"),

```

```

142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153 def _run_startup_checks(config: Any) -> bool:
154     """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155     best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156     cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157     (returns True, ZERO work) when no deployment.profile is selected.
158
159     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160     torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161     WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162     no-op of work, not just of output."""
163     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164     cuda = (
165         probe_cuda_available() if profile else None
166     ) # torch-free on the default-OFF path
167     try:
168         enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169         return True
170     except StartupCheckError:
171         return False # already logged at ERROR by enforce_startup_checks
172
173
174 def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175     """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177     container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179     A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180     from the local 2/3), never a raw traceback:
181     * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
182     running (the shim best-effort cancels it).
183     * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
184     crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185     ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186     from backend.compute import RemoteExecutionFailed
187     from backend.infrastructure.execution_policy import PolicyTimeout
188
189     spec = ComputeJobSpec(
190         video_uri=args.video_uri,
191         out_uri=args.out_uri,
192         segmenter=args.segmenter,
193         config_overrides=tuple(args.set or ()),

```

```

194         skip_validation=bool(getattr(args, "skip_validation", False)),
195     )
196     try:
197         result = compute.run_infer(spec)
198     except RemoteExecutionFailed as e:
199         logger.error(
200             "[worker] remote execution terminated without a completion marker: %s", e
201         )
202         return 5
203     except PolicyTimeout as e:
204         logger.warning(
205             "[worker] remote compute did not complete (no marker within budget): %s", e
206         )
207         return 4
208     logger.info(
209         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210         result.video_id,
211         result.returncode,
212         result.outputs_uri,
213     )
214     return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
227         for a remote
228         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
229         return."""
230         try:
231             marker = {
232                 "video_id": video_id,
233                 "returncode": returncode,
234                 "segment_count": segment_count,
235                 "status": status,
236             }
237             local = os.path.join(scratch, "_done.json")
238             with open(local, "w", encoding="utf-8") as f:
239                 json.dump(marker, f)
240             storage.put(local, done_uri, config=storage_cfg)
241             logger.info("[worker] wrote completion marker -> %s", done_uri)
242         except Exception as e: # never fail a good run because the marker push hiccuped
243             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
244
245     def cmd_infer(args: argparse.Namespace) -> int:
246         config = load_config(args.config) if args.config else load_config()
247         _apply_overrides(config, args.set)
248         if not _run_startup_checks(config):
249             return 2
250
251         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
252         # process.
253         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
254         # through to the
255         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
256         # (the
257         # dispatcher forces compute_target=local_gpu), so it never recursively re-

```

```

dispatches.
254     compute = get_compute_backend(config)
255     if compute is not None:
256         return _dispatch_remote(compute, args)
257
258     storage_cfg = config.storage.model_dump()
259
260     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261     os.makedirs(scratch, exist_ok=True)
262     config.output.output_dir = scratch
263
264     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265     local_video = storage.fetch(
266         args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267     )
268     print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271     video_id = compute_video_id(local_video)
272
273     # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274     # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275     # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276     # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277     # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278     # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279     # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280     # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281     # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282     if getattr(args, "skip_validation", False):
283         print(
284             "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285             "gate and already validated this input."
286         )
287         val_results, val_context = [], {}
288     else:
289         val_results, val_context = validator_registry.run_all_with_context(
290             local_video, config
291         )
292     failures = [r for r in val_results if not r.passed]
293     db = Database(os.path.join(scratch, config.output.db_name))
294     db.add_video(
295         video_id,
296         local_video,
297         "failed" if failures else "processed",
298         [r.model_dump() for r in val_results],
299     )
300
301     def _fail(rc: int) -> int:
302         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
303         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304         # a status="failed" report.json/intervals.csv (0 segments) so the control

```

```

plane's bucket
305         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309         try:
310             _serialize_and_push_outputs(
311                 scratch,
312                 args.out_uri,
313                 video_id,
314                 [],
315                 "failed",
316                 storage_cfg,
317                 str(getattr(args, "video_uri", "") or ""),
318             )
319         except Exception as e: # never mask the real failure on an artifact-push hiccup
320             logger.warning("[worker] could not push failure artifacts: %s", e)
321         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322         if getattr(args, "done_uri", None):
323             _write_done_marker(
324                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
325             )
326         return rc
327
328     segments: List[dict] = []
329     segmenter_actual = None
330     segmentation_ran = False
331     status = "failed"
332     try:
333         if failures:
334             print(
335                 "[worker] validation FAILED: "
336                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
337             )
338             return _fail(2)
339         seg_name = args.segmenter or config.indexing.default_segmenter
340         segmenter_cls = segmenter_registry.get(seg_name)
341         if not segmenter_cls:
342             print(
343                 f"[worker] unknown segmenter: {seg_name!r} "
344                 f"(have {list(segmenter_registry.list_available())})"
345             )
346             return _fail(2)
347         segmenter_actual = seg_name
348         result = segmenter_cls(db, config).process_video(
349             video_id, local_video, max_frames=args.max_frames
350         )
351         segmentation_ran = True
352         if isinstance(result, Failure):
353             print(f"[worker] segmenter FAILED: {result.message}")
354             return _fail(3)
355         segments = result.value if hasattr(result, "value") else result
356         status = "success"
357         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
360         if not segments:
361             detector_impl = (

```

```

362         os.environ.get("RALLY_DETECTOR_IMPL")
363         or getattr(config.indexing, "detector_impl", None)
364         or "auto"
365     )
366     logger.warning(
367         "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
368         "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369         "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
370         "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
371         "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
372         "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
373         "detections, or the input resolution differs from the tuned proxy
resolution. "
374         "Re-run with -v for the native command + frame counts.",
375         video_id,
376         segmenter_actual,
377         detector_impl,
378     )
379     finally:
380         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
381         emit_indexer_report(
382             db=db,
383             video_id=video_id,
384             video_path=local_video,
385             config=config,
386             val_results=val_results,
387             val_context=val_context,
388             segmenter_requested=args.segmenter,
389             segmenter_actual=segmenter_actual,
390             was_reingest=False,
391             segmentation_ran=segmentation_ran,
392         )
393
394     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395     pushed = _serialize_and_push_outputs(
396         scratch,
397         args.out_uri,
398         video_id,
399         segments,
400         status,
401         storage_cfg,
402         str(getattr(args, "video_uri", "") or ""),
403     )
404
405     print(
406         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
407     )
408     for u in pushed:
409         print("    ", u)
410
411     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
412     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
413     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
414     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).

```

```

415         if getattr(args, "done_uri", None):
416             _write_done_marker(
417                 args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418             )
419         return 0
420
421
422     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
423     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
424     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
425
426     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
427     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
428     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
429     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
430
431
432     def _label_clip_name(fname: str) -> str:
433         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
434
435         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
436         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
437
438         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
439         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
440         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
441         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
442
443         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
444         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
445         """
446         name = fname.replace(".rallies.csv", "").replace(".csv", "")
447         return _LABEL_DATE_PREFIX.sub("", name)
448
449
450     def _probe_video_fps_width(path: str):
451         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
452
453         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
454         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
455         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
456         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
457         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
458         rather than guesses.
459         """
460         import json
461         import subprocess
462

```

```

463     try:
464         out = subprocess.check_output(
465             [
466                 ffprobe_bin(),
467                 "-v",
468                 "error",
469                 "-select_streams",
470                 "v:0",
471                 "-show_entries",
472                 "stream=r_frame_rate,width",
473                 "-of",
474                 "json",
475                 path,
476             ],
477             stderr=subprocess.DEVNULL,
478             ).decode()
479         st = (json.loads(out).get("streams") or [{}])[0]
480         num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
481         fps = float(num) / float(den or "1")
482         width = float(st.get("width") or 0)
483         if fps > 0 and width > 0:
484             return fps, width
485     except (
486         subprocess.SubprocessError,
487         ValueError,
488         KeyError,
489         OSError,
490         json.JSONDecodeError,
491     ):
492         pass
493     return None
494
495
496 def _resolve_train_detector(args: argparse.Namespace, config: Any):
497     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
498     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
499     stub=CPU mock). Returns the runner, or prints an error + returns None.
500
501     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
502     the
503     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
504     has
505     no shuttle detector and is rejected.
506     """
507     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
508
509     seg_cls = segmenter_registry.get(args.segmenter)
510     if not seg_cls:
511         print(
512             f"[worker] unknown segmenter: {args.segmenter!r} "
513             f"(have {list(segmenter_registry.list_available())})"
514         )
515         return None
516     seg = seg_cls(None, config)
517     if not isinstance(seg, TrajectoryHybridSegmenter):
518         print(
519             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
520             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
521         )
522         return None
523     try:
524         runner, _ = seg._resolve_runner(config.indexing)
525     except NotImplementedError as e:
526         # e.g. detector_impl='native' for a family without a native runner (tracknet, or

```



```

525         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
526         print(f"[worker] detector not available: {e}")
527         return None
528     ok, msg = runner.healthcheck()
529     if not ok:
530         print(f"[worker] detector environment not ready: {msg}")
531         return None
532     print(f"[worker] detector ready ({args.segmenter}): {msg}")
533     return runner
534
535
536     def cmd_train(args: argparse.Namespace) -> int:
537         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
538         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
539
540         Two trajectory sources (the train pipeline + harness are identical either way):
541         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
542         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
543         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
544         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
545         is the M3.5 "first real training" path; only the trajectory source flips.
546         """
547         import subprocess
548
549         config = load_config(args.config) if args.config else load_config()
550         _apply_overrides(config, args.set)
551         if not _run_startup_checks(config):
552             return 2
553         storage_cfg = config.storage.model_dump()
554
555         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
556         labels_dir = os.path.join(scratch, "labels")
557         traj_dir = os.path.join(scratch, "trajectories")
558         videos_dir = os.path.join(scratch, "videos")
559         os.makedirs(labels_dir, exist_ok=True)
560         os.makedirs(traj_dir, exist_ok=True)
561
562         corpus = args.corpus_uri.rstrip("/")
563         label_uris = sorted(
564             u
565             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
566             if u.lower().endswith(".csv")
567         )
568         if len(label_uris) < 2:
569             print(
570                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
571                 f"under {corpus}/labels/"
572             )
573             return 2
574
575         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
576         runner = None
577         video_by_stem: dict = {}
578         if args.trajectory_source == "detector":
579             os.makedirs(videos_dir, exist_ok=True)
580             runner = _resolve_train_detector(args, config)
581             if runner is None:

```

```

582         return 2
583     for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
584         vname = storage.parse_uri(vuri).name
585         if not vname.lower().endswith(_VIDEO_EXTS):
586             continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
587         video_by_stem[os.path.splitext(vname)[0]] = vuri
588
589     print(f"[worker] trajectory source: {args.trajectory_source}")
590     specs: List[dict] = []
591     for uri in label_uris:
592         fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
593         name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
594         local_label = storage.fetch(
595             uri, os.path.join(labels_dir, fname), config=storage_cfg
596         )
597         if args.trajectory_source == "detector":
598             vuri = video_by_stem.get(name)
599             if not vuri:
600                 print(
601                     f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
602                 )
603                 continue
604             local_video = storage.fetch(
605                 vuri,
606                 os.path.join(videos_dir, storage.parse_uri(vuri).name),
607                 config=storage_cfg,
608             )
609             video_id = compute_video_id(local_video)
610             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
611             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
612             meta = _probe_video_fps_width(local_video)
613             if meta is None:
614                 print(
615                     f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
616                 )
617                 continue
618             fps, frame_width = meta
619             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
620             if not traj:
621                 print(
622                     f"[worker] detector produced no trajectory for {name!r} ? skipping."
623                 )
624                 continue
625             specs.append(
626                 {
627                     "name": name,
628                     "trajectory": traj,
629                     "labels": local_label,
630                     "fps": fps,
631                     "frame_width": frame_width,
632                 }
633             )
634         else:
635             from backend.pipeline.detectors.stub_runner import (
636                 synth_trajectory_from_labels,
637             )
638
639             traj = os.path.join(traj_dir, f"{name}_traj.csv")
640             synth_trajectory_from_labels(
641                 local_label, traj, fps=args.fps, frame_width=args.frame_width
642             )

```

```

643         specs.append(
644             {
645                 "name": name,
646                 "trajectory": traj,
647                 "labels": local_label,
648                 "fps": args.fps,
649                 "frame_width": args.frame_width,
650             }
651         )
652
653     if len(specs) < 2:
654         print(
655             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}]."
656         )
657         return 2
658     manifest_path = os.path.join(scratch, "manifest.json")
659     with open(manifest_path, "w", encoding="utf-8") as f:
660         json.dump(specs, f, indent=2)
661     src_label = (
662         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
663     )
664     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
665
666     out_dir = os.path.join(scratch, "artifacts")
667     cmd = [
668         sys.executable,
669         "-m",
670         "training.gen0.harness",
671         "--manifest",
672         manifest_path,
673         "--out",
674         out_dir,
675         "--version",
676         args.version,
677     ]
678     if args.floor:
679         floor_local = (
680             storage.fetch(
681                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
682             )
683             if "://" in args.floor
684             else args.floor
685         )
686         cmd += ["--floor", floor_local]
687     print("[worker] training:", " ".join(cmd))
688     rc = subprocess.run(cmd).returncode
689     if rc != 0:
690         print(f"[worker] gen0 harness failed (rc={rc})")
691         return rc
692
693     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
694     bundle = os.path.join(out_dir, args.version)
695     out_prefix = args.out_uri.rstrip("/")
696     pushed = []
697     for fn in sorted(os.listdir(bundle)):
698         uri = f"{out_prefix}/weights/{args.version}/{fn}"
699         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
700         pushed.append(uri)
701     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
702     for u in pushed:
703         print("    ", u)
704     return 0
705
706

```

```

707     def build_parser() -> argparse.ArgumentParser:
708         p = argparse.ArgumentParser(
709             prog="python -m backend.worker",
710             description="Storage-aware worker: pull ? run pipeline ? push.",
711         )
712         p.add_argument(
713             "-v",
714             "--verbose",
715             action="store_true",
716             help="DEBUG logging (the native detector command, frame counts, per-stage
717 detail)",
718         )
719         sub = p.add_subparsers(dest="cmd", required=True)
720
721         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
722         pi.add_argument(
723             "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
724         )
725         pi.add_argument(
726             "--out-uri",
727             required=True,
728             help="output prefix (outputs/<video_id>/? is appended)",
729         )
730         pi.add_argument(
731             "--segmenter", default=None, help="default: indexing.default_segmenter"
732         )
733         pi.add_argument(
734             "--config", default=None, help="config.json path (default: ./config.json)"
735         )
736         pi.add_argument(
737             "--scratch", default=None, help="local scratch dir (default: a temp dir)"
738         )
739         pi.add_argument("--max-frames", type=int, default=None)
740         pi.add_argument(
741             "--done-uri",
742             default=None,
743             help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
744             "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
745             "Default-OFF: omit it for the normal local run.",
746         )
747         pi.add_argument(
748             "--set",
749             action="append",
750             default=[],
751             metavar="k=v",
752             help="config override, e.g. --set indexing.skip_ai_handoff=true",
753         )
754         pi.add_argument(
755             "--skip-validation",
756             action="store_true",
757             help="skip the worker's re-validation: the dispatcher (control-plane) already
758 ran the "
759 "validators with its resolved config and only dispatches on PASS, so re-
760 validating here is "
761 "redundant, decodes the video again, and can contradict the gate. Default-OFF:
762 the direct "
763 "`worker infer` CLI still validates.",
764         )
765         pi.set_defaults(func=cmd_infer)
766
767         pt = sub.add_parser(
768             "train",
769             help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
770         )
771         pt.add_argument(

```

```

768         "--corpus-uri",
769         required=True,
770         help="prefix containing labels/ (+ videos/ for "
771         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
772     )
773     pt.add_argument(
774         "--out-uri",
775         required=True,
776         help="output prefix (weights/<version>/ is appended)",
777     )
778     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
779     pt.add_argument(
780         "--trajectory-source",
781         choices=["synth", "detector"],
782         default="synth",
783         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
784         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
785         "(needs videos/ in the corpus; honours detector_impl).",
786     )
787     pt.add_argument(
788         "--segmenter",
789         default="wasb_hybrid",
790         help="trajectory hybrid whose detector is used for --trajectory-source
detector",
791     )
792     pt.add_argument(
793         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
794     )
795     pt.add_argument("--config", default=None)
796     pt.add_argument("--scratch", default=None)
797     pt.add_argument(
798         "--fps",
799         type=float,
800         default=30.0,
801         help="synth trajectory fps (detector probes the video)",
802     )
803     pt.add_argument(
804         "--frame-width",
805         type=float,
806         default=1280.0,
807         help="synth trajectory frame width (detector probes the video)",
808     )
809     pt.add_argument("--set", action="append", default=[], metavar="k=v")
810     pt.set_defaults(func=cmd_train)
811     return p
812
813
814     def _setup_logging(verbose: bool) -> None:
815         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
816         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
817         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
818         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
819         entry), not at import, so importing the module never reconfigures a caller's
logging."""
820         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
821         # configured logging at import), which would leave the level at WARNING and re-
suppress the
822         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
823         logging.basicConfig(

```

```

824         level=logging.DEBUG if verbose else logging.INFO,
825         format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
826         datefmt="%H:%M:%S",
827         force=True,
828     )
829
830
831     def main(argv=None) -> int:
832         args = build_parser().parse_args(argv)
833         _setup_logging(getattr(args, "verbose", False))
834         return args.func(args)
835
836
837     if __name__ == "__main__":
838         sys.exit(main())
839

```

12. assistant (2026-07-07T15:48:58.790Z)

Now let me read the orchestration control-plane caller and the execution_policy semantics.

13. user (2026-07-07T15:49:00.409Z)

```

1      """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3      The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4      Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5      and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6      Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8      SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9      monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11     the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote

```

```

31
32 from backend import storage
33 from backend.config import AppConfig, StitchingConfig
34 from backend.pipeline.segmenters.base import segmenter_registry
35 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36 from backend.pipeline.validators.base import registry
37 from backend.results import Failure
38 from backend.utils.paths import output_base, safe_output_filename
39
40 if (
41     TYPE_CHECKING
42 ): # runtime import would be circular (main imports us for the re-export)
43     from backend.main import ProcessRequest # noqa: F401
44
45 logger = logging.getLogger(__name__)
46
47 # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
48 # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49 # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
50 # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
51 # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52 # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53 _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56 def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
61     keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
62     """
63     if segments:
64         db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
65     else:
66         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
67
68
69 def _validation_failure_message(failures: List[Any]) -> str:
70     details = []
71     for failure in failures:
72         name = getattr(failure, "validator_name", None) or "validation"
73         message = (getattr(failure, "message", None) or "").strip()
74         details.append(f"{name}: {message}" if message else str(name))
75     if not details:
76         return "Video failed validation checks."
77     return "Video failed validation: " + "; ".join(details)
78
79
80 def _report_config_for_input(config: Any, raw_input: str) -> Any:
81     """Return report config that keeps remote-input reports out of disposable
scratch."""
82     storage_config = (
83         config.get("storage")
84         if isinstance(config, dict)
85         else getattr(config, "storage", None)
86     )
87     try:

```

```

88         if storage.input_is_local(raw_input, storage_config):
89             return config
90     except ValueError:
91         return config
92     try:
93         cfg = (
94             config.model_dump(mode="json")
95             if hasattr(config, "model_dump")
96             else dict(config or {})
97         )
98     except Exception:
99         cfg = {}
100     report = dict(cfg.get("report") or {})
101     if report.get("output_dir"):
102         return config
103     report["output_dir"] = os.path.join(output_base(config), "reports")
104     cfg["report"] = report
105     return cfg
106
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
112         the bucket and
113         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
114         call ? so
115         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
116         were produced.
117         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
118         pattern (cloud
119         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
120         malformed row is
121         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
122         temp dir is
123         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
124         a job failure
125         and prunes any partial rows (destroy-AFTER-confirm)."""
126         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
127         ref = storage.parse_uri(uri)
128         n = 0
129         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
130             local = storage.fetch(
131                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
132             )
133             with open(local, newline="", encoding="utf-8") as f:
134                 for row in csv.DictReader(f):
135                     if n >= max_rows:
136                         logger.warning(
137                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
138                             rest.",
139                             max_rows,
140                             uri,
141                         )
142                     )
143                     break
144                 try:
145                     start, end = float(row["start"]), float(row["end"])
146                 except (KeyError, TypeError, ValueError):
147                     continue
148                 if not (math.isfinite(start) and math.isfinite(end)):
149                     continue # reject inf/nan times rather than persist a junk rally
150                 meta: Dict[str, Any] = {
151                     "source": "cloud_run",
152                     "ending_reason": (row.get("ending_reason") or "").strip(),
153                 }

```



```

145         sc = (row.get("shot_count") or "").strip()
146         if sc:
147             try:
148                 meta["shot_count"] = int(
149                     float(sc)
150                 ) # OverflowError on 'inf'/'1e400'
151             except (ValueError, OverflowError):
152                 meta["shot_count"] = sc
153         db.add_play_segment(video_id, start, end, metadata=meta)
154         n += 1
155     return n
156
157
158     def _cloud_available(cfg) -> bool:
159         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
160 3).
161
162         True when the control plane is wired for Cloud Run: either the configured default
163         target is already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
164         coordinates (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
165         (``storage.output_root``)
166         are present so a forced per-request override can actually dispatch + collect a
167         result.
168
169         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
170         would work."""
171         deployment = getattr(cfg, "deployment", None)
172         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
173             return True
174         has_job = bool(
175             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
176         )
177         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
178         return bool(has_job and out_root)
179
180
181     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
182         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
183         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
184         comes
185         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
186         "starting"
187         (queue/cold-start), so the very first tick already shows movement."""
188         minutes = int(elapsed_sec) // 60
189         if minutes < 1:
190             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
191         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
192
193
194     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
195         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
196         gs://
197         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
198         keep
199         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
200         no
201         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
202         if getattr(input_ref, "backend", "") != "gcs":
203             return None
204         sc = _storage_settings(cfg)
205         if not sc:
206             return None
207         try:

```

```

199         st = storage.get_storage("gcs", config=sc).stat(input_ref)
200         return getattr(st, "md5", None) or None
201     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
202         logger.info(
203             "[API] submit-time md5 unavailable for %s (metadata stat failed)",
204             getattr(input_ref, "uri", input_ref),
205             exc_info=True,
206         )
207     return None
208
209
210 def _cached_process_payload(
211     video_id: str, validation_results, segments, requested_compute
212 ) -> Dict[str, Any]:
213     """The /api/process result for a video whose detection already exists ? ONE shape
214     for
215     both producers (the drain's DB-cache short-circuit and the submit-time cache,
216     #484)."""
217     return {
218         "video_id": video_id,
219         "status": "success",
220         "message": (
221             f"Reused {len(segments)} cached play segments. "
222             "Pass force=true to reprocess this video."
223         ),
224         "results": validation_results or [],
225         "play_segments": segments,
226         "cached": True,
227         "compute": {"path": "cached", "requested": requested_compute},
228     }
229
230 def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
231     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
232     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
233     decide."""
234     ref = storage.parse_uri(uri)
235     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
236         local = storage.fetch(
237             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
238         )
239         with open(local, encoding="utf-8") as f:
240             loaded = json.load(f)
241         return loaded if isinstance(loaded, dict) else None
242
243 # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
244 hits
245 # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
246 # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
247 instance
248 # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
249 _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
250 _REHYDRATE_LOCKS_GUARD = threading.Lock()
251
252 def _rehydrate_lock(video_id: str) -> threading.Lock:
253     with _REHYDRATE_LOCKS_GUARD:
254         return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
255
256 def probe_process_cache(db, cfg, video_id: str) -> bool:
257     """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
258     bucket)?
259

```

```

258     Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
259     ordering). Used to decide whether a submit can skip the local-scratch fetch guard
260     (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
261     errors report False (? the normal dispatch path with its guards)."""
262     try:
263         existing = db.get_video(video_id)
264         if (
265             existing
266             and existing.get("status") == "processed"
267             and db.query_play_segments(video_id, {})
268         ):
269             return True
270         sc = _storage_settings(cfg)
271         if not sc or (sc.get("backend") or "local") == "local":
272             return False
273         output_root = (sc.get("output_root") or "").rstrip("/")
274         if not output_root:
275             return False
276         report_uri = f"{output_root}/outputs/{video_id}/report.json"
277         if not storage.exists(report_uri, config=sc):
278             return False
279         report = _read_bucket_json(report_uri, sc)
280         return bool(report and report.get("status") == "success")
281     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
282         logger.warning(
283             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
284         )
285     return False
286
287
288     def cached_process_result(
289         db, cfg, video_id: str, video_path: str, requested_compute=None
290     ) -> Optional[Dict[str, Any]]:
291         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
292         the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
293         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
294         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
295         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
296         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
297         partial earlier ingest is replaced, never double-counted.
298
299         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
300         per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
301         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
302         job before invoking (writes happen post-claim)."""
303     try:
304         with _rehydrate_lock(video_id):
305             existing = db.get_video(video_id)
306             if existing and existing.get("status") == "processed":
307                 segs = db.query_play_segments(video_id, {})
308                 if segs:
309                     return _cached_process_payload(
310                         video_id,
311                         existing.get("validation_results", []),
312                         segs,
313                         requested_compute,

```

```

314         )
315         sc = _storage_settings(cfg)
316         if not sc or (sc.get("backend") or "local") == "local":
317             return None
318         output_root = (sc.get("output_root") or "").rstrip("/")
319         if not output_root:
320             return None
321         outputs_uri = f"{output_root}/outputs/{video_id}"
322         report_uri = f"{outputs_uri}/report.json"
323         if not storage.exists(report_uri, config=sc):
324             return None
325         report = _read_bucket_json(report_uri, sc)
326         if not report or report.get("status") != "success":
327             return None
328         if not video_path:
329             # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
330             # carries the worker's --video-uri (P3); older reports fall back to the
331             # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
332             # recover, and the next same-md5 submit re-binds the truthful path via
333             # add_video's upsert.
334             video_path = str(report.get("video_uri") or "") or _first_object_under(
335                 f"{output_root}/videos/{video_id}/", sc
336             )
337         play_wm, cand_wm = db.segment_watermarks(video_id)
338         db.add_video(video_id, video_path, "processed", [])
339         n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
340         if n <= 0:
341             return None
342         segments = [
343             s
344             for s in db.query_play_segments(video_id, {})
345             if int(s.get("id", 0)) > play_wm
346         ]
347         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
348         if not segments:
349             return None
350         logger.info(
351             "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
352             n,
353             video_id,
354             outputs_uri,
355         )
356         return _cached_process_payload(video_id, [], segments, requested_compute)
357     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
358         logger.warning(
359             "[API] submit-time cache check failed for %s ? dispatching normally",
360             video_id,
361             exc_info=True,
362         )
363     return None
364
365
366     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
367         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
368         try:
369             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
370         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
371             return ""
372
373
374     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
375

```

```

376
377 def read_path_rehydrate(db, cfg, video_id: str) -> bool:
378     """"#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
379     nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
380     so
381     ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
382     DB
383     has the video's rows now (re-query). Non-md5 ids return False without touching the
384     bucket; every failure degrades to False (the read just reports what the DB has).""
385     vid = (video_id or "").strip().lower()
386     if not _MD5_HEX_RE.fullmatch(vid):
387         return False
388     if not probe_process_cache(db, cfg, vid):
389         return False
390     return cached_process_result(db, cfg, vid, "") is not None
391
392 def _maybe_dispatch_remote(
393     cfg,
394     db,
395     req: "ProcessRequest",
396     video_id: str,
397     seg_name: str,
398     val_results,
399     play_wm: int,
400     cand_wm: int,
401     notify,
402 ) -> Optional[Tuple[Dict[str, Any], bool]]:
403     """"Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
404     ``deployment.compute_target
405     == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
406     intervals into
407     THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
408     Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
409     the cloud job
410     actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
411     observability report
412     doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
413     process segmenter
414     (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
415     non-``cloud_run`` target, so
416     the in-process path stays byte-identical).""
417     from backend.compute import ComputeJobSpec, get_compute_backend
418
419     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
420         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
421         (id > play_wm);
422         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
423         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
424         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
425         only when the
426         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
427         rc / ingest
428         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
429         records the
430         # intended backend, `requested` the picker choice, `dispatched` whether the
431         remote job ran.
432         return (
433             {
434                 "video_id": video_id,
435                 "status": "failed",
436                 "message": msg,
437                 "results": [r.model_dump() for r in val_results],
438                 "play_segments": [],

```

```

428         "compute": {
429             "path": "cloud_run" if ran else "not_run",
430             "requested": req.compute,
431             "target": "cloud_run",
432             "dispatched": ran,
433         },
434     },
435     ran,
436 )
437
438     # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
439     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
440     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
441     choice = (req.compute or "").strip().lower()
442     if choice and choice not in ("local", "cloud"):
443         logger.warning(
444             "[API] ignoring unknown compute=%r (expected 'local'|"cloud'); using server
default.",
445             req.compute,
446         )
447     choice = ""
448     if choice == "local":
449         return None # explicit "run on my machine" ? in-process regardless of the
server's target
450     if choice == "cloud":
451         if not _cloud_available(cfg):
452             return _failed(
453                 "cloud-serving was requested but this server isn't configured for it "
454                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
455                 False,
456             )
457     compute = get_compute_backend(cfg, target_override="cloud_run")
458 else:
459     compute = get_compute_backend(
460         cfg
461     ) # server default (default-OFF unless cloud_run)
462 if compute is None:
463     return None # default-OFF ? run the in-process segmenter below (unchanged)
464
465     storage_cfg = getattr(cfg, "storage", None)
466
467     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
468     try:
469         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
470     except ValueError as e:
471         return _failed(
472             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
473         )
474     if input_ref.backend == "local":
475         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
476         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
477         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
478         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
479         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
480         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left

```

```

at its
481         # default, while a bare/local-looking root can't stage no matter what
input_backend says
482         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
483         # gives us nothing to build the staged URI under).
484         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
485         try:
486             root_backend = (
487                 storage.parse_uri(input_root).backend if input_root else "local"
488             )
489         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
490             root_backend = "local"
491         if root_backend == "local":
492             return _failed(
493                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
494                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
495                 "Run job.",
496                 False,
497             )
498         local_src = input_ref.key # resolved local path of the upload
499         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
500         try:
501             notify("staging upload to bucket")
502             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
503             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
504             storage.put(
505                 local_src,
506                 staged_uri,
507                 config=(
508                     storage_cfg.model_dump() if storage_cfg is not None else None
509                 ),
510             )
511         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
512             logger.error(
513                 "[API] cloud-serving: failed to stage %s -> %s: %s",
514                 local_src,
515                 staged_uri,
516                 e,
517                 exc_info=True,
518             )
519             return _failed(
520                 f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
521                 False,
522             )
523         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
524         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
525         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
526         req.video_path = staged_uri
527         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
528         db.add_video(
529             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
530         )
531         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()

```

```

532         if not out_root:
533             return _failed(
534                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
535                 "? that's where the Cloud Run job writes the result.",
536                 False,
537             )
538
539         notify("dispatching (cloud_run)")
540         logger.info(
541             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
542             req.video_path,
543             out_root,
544         )
545         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
546         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
547         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
548         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
549         overrides = []
550         idx_cfg = getattr(cfg, "indexing", None)
551         sk = getattr(idx_cfg, "skip_ai_handoff", None)
552         if sk is not None:
553             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
554         # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
555         # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
556         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
557         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
558         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
559         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
560         wasb_cfg = getattr(idx_cfg, "wasb", None)
561         if wasb_cfg is not None:
562             decode_backend = getattr(wasb_cfg, "decode_backend", None)
563             if decode_backend:
564                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565             batch_size = getattr(wasb_cfg, "batch_size", None)
566             if batch_size:
567                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
568             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569             if prefetch:
570                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
572         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
573         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
574         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
575         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
576         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
577         spec = ComputeJobSpec(
578             video_uri=req.video_path,
579             out_uri=out_root,
580             segmenter=seg_name,

```



```

581         config_overrides=tuple(overrides),
582         skip_validation=True,
583     )
584     try:
585         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
586         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
587         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
588         # also gives the SPA a stall signal to time out on, instead of a blind wall
589         # clock.
590         result = compute.run_infer(
591             spec,
592             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
593         )
594     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
595         failure, not a 500
596         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
597         return _failed(f"cloud dispatch error: {e}", True)
598     if result.returncode != 0:
599         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
600     if result.video_id and result.video_id != video_id:
601         # The worker re-hashes the bucket bytes; a divergence means the input changed
602         # between this
603         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
604         # diagnosable.
605         logger.warning(
606             "[API] cloud video_id %s != local %s ? input bytes differ between the API
607             hash "
608             "and the worker hash; ingesting under the local id.",
609             result.video_id,
610             video_id,
611         )
612     notify("ingesting (cloud_run)")
613     storage_dict = (
614         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
615         can't narrow
616         if hasattr(storage_cfg, "model_dump")
617         else {}
618     )
619     try:
620         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
621     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
622         500 or leak rows
623         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
624         return _failed(
625             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
626             True,
627         )
628     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
629     # drops the old ones.
630     segments = [
631         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
632     ]
633     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
634     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
635     # is the real
636     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
637     # <execution>).
638     provenance = {
639         "path": "cloud_run",
640         "requested": req.compute,
641         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
642         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),

```

```

635         "execution": result.execution,
636         "outputs_uri": result.outputs_uri,
637     }
638     logger.info(
639         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
640         result.execution,
641         provenance["job"],
642         provenance["region"],
643         result.outputs_uri,
644         video_id,
645     )
646     return (
647         {
648             "video_id": video_id,
649             "status": "success",
650             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
651             "results": [r.model_dump() for r in val_results],
652             "play_segments": segments,
653             "compute": provenance,
654         },
655         True,
656     )
657
658
659 def _execute_processing(
660     config=None, db=None, req=None, progress=None
661 ) -> Dict[str, Any]:
662     """The full hash ? validate ? segment ? report pipeline for one video.
663
664     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
665     module-level ``global_config``/``db_instance`` (backward-compatible with existing
666     tests that pass only ``req``). Route handlers always pass explicit params; older
667     test code that calls ``_execute_processing(req)`` still works.
668
669     ``progress`` is an optional callable(stage: str) used to surface coarse job
670 progress.
671
672     Raises on unexpected internal errors ? the caller records those as a job failure
673 """
674 import backend.main as _main # call-time seam: main's globals/model stay patchable
675
676 # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
677 # shift positional args ? req arrives in position 0 (config), progress in position 1
678 (db).
679
680 if config is not None and not isinstance(config, AppConfig):
681     # Old signature: _execute_processing(req, progress=None)
682     if req is None and db is not None and isinstance(db, _main.ProcessRequest):
683         req = db
684         db = None
685     elif req is None and isinstance(config, _main.ProcessRequest):
686         req = config
687         config = None
688
689 # Resolve config/db from module globals when not passed explicitly
690 if config is None:
691     config = _main.global_config
692 if db is None:
693     db = _main.db_instance
694
695 notify = progress or (lambda stage: None)
696
697 notify("hashing")
698 # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
699 (identity ?
700 # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The

```

```

original
695         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
696         # consumers (hash / validators / segmenter) use the resolved local path.
697         local_video = ""
698         try:
699             local_video = storage.acquire_local_input(
700                 req.video_path, getattr(config, "storage", None)
701             )
702             video_id = _main.compute_video_id(local_video)
703         except Exception:
704             storage.cleanup_acquired_local_input(
705                 req.video_path, local_video, getattr(config, "storage", None)
706             )
707             raise
708         try:
709             existing_video = db.get_video(video_id)
710         except Exception:
711             storage.cleanup_acquired_local_input(
712                 req.video_path, local_video, getattr(config, "storage", None)
713             )
714             raise
715         was_reingest = existing_video is not None
716
717         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
718         default_seg = getattr(
719             getattr(config, "indexing", None), "default_segmenter", "motion"
720         )
721         seg_name = req.segmenter or default_seg
722         segmenter_actual = None
723         segmentation_ran = False
724         val_results: List[Any] = []
725         val_context: Dict[str, Any] = {}
726         # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
727         # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
728         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
729         compute_actual: Optional[str] = None
730         response: Optional[Dict[str, Any]] = None
731         try:
732             if (
733                 existing_video
734                 and existing_video.get("status") == "processed"
735                 and not req.force
736             ):
737                 cached_segments = db.query_play_segments(video_id, {})
738                 if cached_segments:
739                     logger.info(
740                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
741                         video_id,
742                     )
743                 response = _cached_process_payload(
744                     video_id,
745                     existing_video.get("validation_results", []),
746                     cached_segments,
747                     getattr(req, "compute", None),
748                 )
749                 return response
750
751         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report)
752         notify("validating")

```

```

753     logger.info(f"Running validations for {req.video_path}...")
754     val_results, val_context = registry.run_all_with_context(local_video, config)
755     failures = [r for r in val_results if not r.passed]
756
757     if failures:
758         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
759         # status; it no longer destroys the video's prior good segments.
760         db.add_video(
761             video_id,
762             req.video_path,
763             "failed",
764             [r.model_dump() for r in val_results],
765         )
766         response = {
767             "video_id": video_id,
768             "status": "failed",
769             "message": _validation_failure_message(failures),
770             "results": [r.model_dump() for r in val_results],
771         }
772         return response
773
774     # 2. Run segmenter & indexer
775     logger.info("[API] Video passed checks. Segmenting and indexing...")
776     db.add_video(
777         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
778     )
779
780     segmenter_cls = segmenter_registry.get(seg_name)
781     if not segmenter_cls:
782         # Submit-time validation already 400s unknown names; this is the defensive
783         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
784         available = list(segmenter_registry.list_available().keys())
785         response = {
786             "video_id": video_id,
787             "status": "failed",
788             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
789             "results": [r.model_dump() for r in val_results],
790             "play_segments": [],
791         }
792         return response
793
794     logger.info(f"[API] Using segmenter: {seg_name}")
795     notify(f"segmenting ({seg_name})")
796     segmenter_actual = seg_name
797     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
798     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
799     play_wm, cand_wm = db.segment_watermarks(video_id)
800     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
801     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
802     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
803     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
804     # they're dropped on the cloud path in this slice ? a documented follow-up.)
805     remote = _maybe_dispatch_remote(
806         config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
807     )
808     if remote is not None:
809         response, segmentation_ran = (

```

```

810         remote # `ran` = did the cloud job actually execute (report)
811     )
812     return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
813     # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
814     compute_actual = "in_process"
815     segmenter = segmenter_cls(db, config)
816     try:
817         # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
818         # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
819         with _LOCAL_COMPUTE_LANE:
820             result = segmenter.process_video(
821                 video_id, local_video, req.player_pool, req.max_frames
822             )
823     except Exception:
824         # A raising segmenter must not leave half-written rows: restore the pre-run
825         # state (same destroy-after-confirm contract as the Failure branch), then
let
826         # the job worker record the failure.
827         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
828         raise
829     segmentation_ran = True
830
831     if isinstance(result, Failure):
832         logger.error(f"[API] Segmenter failed: {result.message}")
833         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
834         response = {
835             "video_id": video_id,
836             "status": "failed",
837             "message": f"Segmenter '{seg_name}' failed: {result.message}",
838             "results": [r.model_dump() for r in val_results],
839             "play_segments": [],
840         }
841         return response
842
843     segments = result.value if hasattr(result, "value") else result
844     notify("finalizing")
845     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
846
847     response = {
848         "video_id": video_id,
849         "status": "success",
850         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value
851         "results": [r.model_dump() for r in val_results],
852         "play_segments": segments,
853     }
854     return response
855 finally:
856     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
857     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
858     # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
859     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
860     if isinstance(response, dict):
861         response.setdefault(
862             "compute",
863             {

```

```

864         "path": compute_actual or "not_run",
865         "requested": getattr(req, "compute", None),
866     },
867 )
868 prov = response.get("compute", {}) if isinstance(response, dict) else {}
869 logger.info(
870     "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
871     video_id,
872     getattr(req, "compute", None),
873     prov.get("path"),
874     prov.get("dispatched"),
875     (response or {}).get("status") if isinstance(response, dict) else None,
876 )
877 # Always emit the per-video observability report (next to the video, or to
878 # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
879     # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
880     # doesn't surface arbitrary run signals.)
881     paths = _main.emit_indexer_report(
882         db=db,
883         video_id=video_id,
884         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
885         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
886         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
887         # still preserved in DB provenance via db.add_video.
888         video_path=local_video,
889         config=_report_config_for_input(config, req.video_path),
890         val_results=val_results,
891         val_context=val_context,
892         segmenter_requested=req.segmenter,
893         segmenter_actual=segmenter_actual,
894         was_reingest=was_reingest,
895         segmentation_ran=segmentation_ran,
896     )
897 if paths and isinstance(response, dict):
898     response["reports"] = paths
899 storage.cleanup_acquired_local_input(
900     req.video_path, local_video, getattr(config, "storage", None)
901 )
902
903
904 def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
905     """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
906
907     Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
908     missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
909     to a no-op rather than blocking a job it cannot measure."""
910     try:
911         sc = (
912             storage_config.model_dump()
913             if hasattr(storage_config, "model_dump")
914             else (storage_config or {})
915         )
916         backend = storage.get_storage(input_ref.backend, config=sc)
917         return int(backend.stat(input_ref).size)
918     except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block
919         logger.debug(
920             "free-space: could not stat remote source %s: %s", input_ref.uri, e

```

```

921         )
922         return None
923
924
925     class _CompileError(Exception):
926         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
927         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
928
929         def __init__(self, status_code: int, detail: str):
930             super().__init__(detail)
931             self.status_code = status_code
932             self.detail = detail
933
934
935     def _resolve_compile(
936         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
937     ):
938         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
939         row could change between submit and run). Verifies the video exists, the requested
segments match
940         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
941         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
942         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
943         video = db.get_video(video_id)
944         if not video:
945             raise _CompileError(404, "Indexed video record not found.")
946         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
947         try:
948             out_name = safe_output_filename(output_filename)
949         except ValueError as e:
950             raise _CompileError(400, str(e)) from e
951
952         all_segments = db.query_play_segments(video_id, {})
953         matched = [s for s in all_segments if s["id"] in segment_ids]
954         if not matched:
955             raise _CompileError(400, "No matching play segments found to stitch.")
956
957         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
958         kept = []
959         for s in matched:
960             meta = s.get("metadata") or {}
961             sc = meta.get("shot_count") if isinstance(meta, dict) else None
962             try:
963                 if sc is not None and int(sc) < stitching.min_shot_count:
964                     continue
965             except (TypeError, ValueError):
966                 pass # unparseable count -> treat as unknown, keep
967             kept.append(s)
968         if len(kept) < len(matched):
969             logger.info(
970                 "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
971                 stitching.min_shot_count,
972                 len(matched) - len(kept),
973                 len(matched),
974             )
975         if not kept:
976             raise _CompileError(

```

```

977         400,
978         f"All {len(matched)} matching segments fall below "
979         f"stitching.min_shot_count={stitching.min_shot_count}.".",
980     )
981     return video, kept, out_name
982
983
984     def reel_object_uri(cfg, video_id: str, name: str) -> str:
985         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
986         (#463), or "" when
987         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
988         compile
989         mirror-upload and the /api/highlights redirect:
990         ``<output_root>/highlights/<video_id>/<name>``."""
991         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
992         if not root or "://" not in root:
993             return ""
994         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
995
996
997     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
998         """The active storage settings dict (per-backend config for the storage layer ? S3
999         endpoint/region,
1000         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1001         default backend
1002         isn't silently constructed as a fresh default."""
1003         sc = getattr(cfg, "storage", None)
1004         return sc.model_dump() if sc is not None else None
1005
1006
1007     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1008         """A short-lived signed URL for a reel mirrored to the output bucket, so
1009         /api/highlights can
1010         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
1011         ephemeral local
1012         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
1013         config for BOTH
1014         the existence check and the signing (extracted here to keep backend/main.py lean ?
1015         P23/P24)."""
1016         reel_uri = reel_object_uri(cfg, video_id, name)
1017         if not reel_uri:
1018             return None
1019         settings = _storage_settings(cfg)
1020         if storage.exists(reel_uri, config=settings):
1021             return storage.signed_url(reel_uri, config=settings, method="GET")
1022         return None
1023
1024
1025     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1026         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
1027         stitch the reel,
1028         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1029         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
1030         stitch error)
1031         so the SPA always gets a clean verdict instead of a worker-crash error."""
1032         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
1033         backend.main
1034
1035         notify = progress or (lambda stage: None)
1036         params = json.loads(job.get("params") or "{}")
1037         video_id = job.get("video_id") or ""
1038         segment_ids = params.get("segment_ids") or []
1039         locale = params.get("locale")
1040
1041         notify("resolving")

```



```

1030     try:
1031         video, kept, out_name = _resolve_compile(
1032             db, cfg, video_id, segment_ids, params.get("output_filename", "")
1033         )
1034     except _CompileError as e:
1035         return {"status": "failed", "message": e.detail, "video_id": video_id}
1036
1037     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1038     output_dir = (
1039         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1040     )
1041     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1042     # unknown/absent locale keeps the configured default unchanged.
1043     localized_watermark = localize_watermark(stitching.watermark, locale)
1044     compiler = _main.VideoCompiler(
1045         output_dir,
1046         begin_padding=stitching.begin_padding,
1047         end_padding=stitching.end_padding,
1048         watermark=localized_watermark,
1049     )
1050     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1051
1052     notify("stitching")
1053     local_src = ""
1054     try:
1055         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1056         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1057         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1058         local_src = storage.acquire_local_input(
1059             video["filepath"], getattr(cfg, "storage", None)
1060         )
1061         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1062         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1063         with _LOCAL_COMPUTE_LANE:
1064             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1065     except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1066         logger.error(
1067             "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1068         )
1069         return {
1070             "status": "failed",
1071             "message": f"Highlight stitching failed: {e}",
1072             "video_id": video_id,
1073         }
1074     finally:
1075         if local_src:
1076             storage.cleanup_acquired_local_input(
1077                 video["filepath"], local_src, getattr(cfg, "storage", None)
1078             )
1079
1080     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1081     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1082     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1083     # still falls back to the local file on the same instance).
1084     reel_uri = reel_object_uri(cfg, video_id, out_name)

```

```

1085         if reel_uri:
1086             try:
1087                 notify("uploading reel")
1088                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1089                 # project/client) is used, not a fresh default.
1090                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1091             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1092                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1093
1094         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1095         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1096         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1097         notify("finished")
1098         return {
1099             "status": "success",
1100             "filepath": out_path,
1101             "download_url": download_url,
1102             "video_id": video_id,
1103         }
1104

```

14. assistant (2026-07-07T15:49:25.018Z)

Now let me read the execution_policy to understand poll_until semantics, and check the test helpers.

15. user (2026-07-07T15:49:26.211Z)

```

1      """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3      Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4      - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5      for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6      so it is abandoned, not joined ? it never blocks the process.
7      - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8      ``max_wait_sec`` deadline).
9
10     **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11     waiting (a fake clock advances instantly).
12
13     **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14     transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15     ``execution.policy`` logger to watch the poll trail.
16     """
17
18     from __future__ import annotations
19
20     import logging
21     import threading
22     import time as _time
23     from dataclasses import asdict, dataclass
24     from enum import Enum
25     from typing import Callable, Optional, TypeVar
26
27     T = TypeVar("T")

```

```

28
29     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
30     LOG = logging.getLogger("execution.policy")
31
32
33     class WaitMode(str, Enum):
34         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
35
36         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
37         ABORT = "abort" # one attempt; on hang/fail, stop immediately
38         BLOCK = "block" # bounded in-process blocking wait (no reschedule)
39
40
41     class PolicyTimeout(TimeoutError):
42         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
43
44
45     @dataclass(frozen=True)
46     class ExecutionPolicy:
47         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
48
49         mode: WaitMode = WaitMode.WAIT_AND_RESUME
50         poll_every_n_sec: float = 10.0
51         op_timeout_sec: float = (
52             120.0 # HARD per-attempt deadline ? the hang-killer that was missing
53         )
54         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
55         max_retries: int = 5 # retries (=> max_retries+1 attempts) for failures/hangs
56         backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
57         on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by P2/P3)
58         resumable: bool = True
59
60         def to_dict(self) -> dict:
61             d = asdict(self)
62             d["mode"] = self.mode.value
63             return d
64
65         @classmethod
66         def from_dict(cls, d: dict) -> "ExecutionPolicy":
67             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
68             kw = {k: v for k, v in d.items() if k in known}
69             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
70                 kw["mode"] = WaitMode(kw["mode"])
71             return cls(**kw)
72
73
74     #: The sane default (also what a job gets if it declares no policy).
75     DEFAULT_POLICY = ExecutionPolicy()
76
77
78     def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
79         """Deterministic exponential backoff (no jitter ? reproducible tests)."""
80         return policy.backoff_base_sec * (2**attempt)
81
82
83     def run_bounded(
84         fn: Callable[[], T],
85         policy: ExecutionPolicy = DEFAULT_POLICY,
86         *,
87         label: str = "op",

```

```

88         logger: Optional[logging.Logger] = None,
89         sleep: Callable[[float], None] = _time.sleep,
90     ) -> T:
91         """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
92
93         Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
94         last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
95         """
96         log = logger or LOG
97         attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
98         last_exc: Optional[BaseException] = None
99         for attempt in range(attempts):
100             box: dict = {}
101             done = threading.Event()
102
103             # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
104             # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
105             def _runner(_box: dict = box, _done: threading.Event = done) -> None:
106                 try:
107                     _box["val"] = fn()
108                 except (
109                     BaseException
110                 ) as e: # relay ANY error (incl. timeouts) to the caller thread
111                     _box["exc"] = e
112                 finally:
113                     _done.set()
114
115             threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
116             if done.wait(timeout=policy.op_timeout_sec):
117                 if "exc" in box:
118                     last_exc = box["exc"]
119                     log.info(
120                         "%s: attempt %d/%d failed: %s",
121                         label,
122                         attempt + 1,
123                         attempts,
124                         last_exc,
125                     )
126                 else:
127                     if attempt:
128                         log.info(
129                             "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
130                         )
131                     return box["val"] # type: ignore[return-value]
132             else:
133                 last_exc = PolicyTimeout(
134                     f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
135                 )
136                 log.warning(
137                     "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
138                     label,
139                     attempt + 1,
140                     attempts,
141                     policy.op_timeout_sec,
142                 )
143             if attempt < attempts - 1:
144                 wait = _backoff_sec(policy, attempt)
145                 log.debug("%s: backing off %.1fs before retry", label, wait)
146                 sleep(wait)
147         assert last_exc is not None

```

```

148         log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
149         raise last_exc
150
151
152     def poll_until(
153         check: Callable[[], Optional[T]],
154         policy: ExecutionPolicy = DEFAULT_POLICY,
155         *,
156         label: str = "op",
157         logger: Optional[logging.Logger] = None,
158         sleep: Callable[[float], None] = _time.sleep,
159         monotonic: Callable[[], float] = _time.monotonic,
160         on_tick: Optional[Callable[[float], None]] = None,
161     ) -> T:
162         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
163         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
164
165         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
166         ``on_tick(elapsed_sec)`` (optional) fires once per still-waiting iteration ? the
caller's live
167         progress heartbeat (#482). Best-effort by contract: a raising callback is swallowed
168         (DEBUG-logged), never allowed to kill the wait it reports on.
169         """
170         log = logger or LOG
171         start = monotonic()
172         polls = 0
173         log.info(
174             "%s: waiting (poll_every=%0.0fs, max_wait=%0.0fs)",
175             label,
176             policy.poll_every_n_sec,
177             policy.max_wait_sec,
178         )
179         while True:
180             result = check()
181             if result is not None:
182                 log.info(
183                     "%s: ready after %d poll(s) / %0.0fs", label, polls, monotonic() - start
184                 )
185                 return result
186             elapsed = monotonic() - start
187             if elapsed >= policy.max_wait_sec:
188                 log.warning(
189                     "%s: still not ready after %0.0fs (max_wait) ? giving up", label, elapsed
190                 )
191                 raise PolicyTimeout(
192                     f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
193                 )
194             polls += 1
195             log.debug("%s: poll %d, %0.0fs elapsed, not ready", label, polls, elapsed)
196             if on_tick is not None:
197                 try:
198                     on_tick(elapsed)
199                 except Exception: # noqa: BLE001 ? progress reporting must never kill the
wait
200                     log.debug("%s: on_tick callback failed", label, exc_info=True)
201             sleep(policy.poll_every_n_sec)
202

```

16. assistant (2026-07-07T15:51:06.871Z)

Let me read the test helpers to understand the fakes, and check git integrity of the rebase.

17. user (2026-07-07T15:51:08.393Z)

```
1      """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3      All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4      worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5      ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6      selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7      """
8
9      import json
10     import os
11     import tempfile
12
13     import pytest
14
15     from backend.compute import (
16         ComputeJobSpec,
17         RemoteExecutionFailed,
18         get_compute_backend,
19     )
20     from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
21     from backend.config import AppConfig
22     from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout
23
24     # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).
25     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
26
27
28     class _FakeRunClient(CloudRunJobClient):
29         """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
local bucket
30         exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""
31
32         def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
33             self.calls: list[dict] = []
34             self.video_id, self.segment_count, self.returncode = (
35                 video_id,
36                 segment_count,
37                 returncode,
38             )
39
40         def run_job(self, job_name, argv, *, region, project=None):
41             self.calls.append(
42                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
43             )
44             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
45             done_uri = argv[argv.index("--done-uri") + 1]
46             outputs = os.path.join(out_uri, "outputs", self.video_id)
47             os.makedirs(outputs, exist_ok=True)
48             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
49                 json.dump(
50                     {
51                         "video_id": self.video_id,
52                         "segment_count": self.segment_count,
53                         "status": "success",
54                     },
55                     f,
56                 )
```

```

57         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
58         with open(done_uri, "w", encoding="utf-8") as f:
59             json.dump(
60                 {
61                     "video_id": self.video_id,
62                     "returncode": self.returncode,
63                     "segment_count": self.segment_count,
64                     "status": "success",
65                 },
66                 f,
67             )
68         return "exec-123"
69
70     def cancel_execution(self, execution):
71         raise AssertionError(
72             "cancel must never fire on the happy path"
73         ) # pragma: no cover
74
75
76     class _NoopRunClient(CloudRunJobClient):
77         """Records the dispatch but does NO filesystem writes ? for tests that fake the
storage layer
78         directly (a gs:// out_uri has no local dir to makedirs)."""
79
80         def __init__(self):
81             self.calls: list[dict] = []
82             self.cancelled: list[str] = []
83
84         def run_job(self, job_name, argv, *, region, project=None):
85             self.calls.append(
86                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
87             )
88             return "exec"
89
90         def cancel_execution(self, execution):
91             self.cancelled.append(execution)
92
93
94     class _NeverDoneClient(CloudRunJobClient):
95         """Simulates a still-running execution: dispatch succeeds (returning a fully-
qualified
96         execution name) but NO done-marker is ever written, so the bucket-poll must time
out."""
97
98         EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-
slow"
99
100         def __init__(self, *, cancel_error: Exception | None = None):
101             self.calls: list[dict] = []
102             self.cancelled: list[str] = []
103             self._cancel_error = cancel_error
104
105         def run_job(self, job_name, argv, *, region, project=None):
106             self.calls.append(
107                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
108             )
109             return self.EXEC_NAME
110
111         def cancel_execution(self, execution):
112             self.cancelled.append(execution)
113             if self._cancel_error is not None:
114                 raise self._cancel_error
115
116
117     class _CrashedNoMarkerClient(CloudRunJobClient):

```

```

118         """Simulates the bug's failure class: the worker execution TERMINATES (crash / OOM /
an old
119         image argparse-aborting on an unknown flag) WITHOUT ever writing a done-marker.
Dispatch returns
120         a resolvable execution name, no marker is ever written, and the Executions API
reports a terminal
121         state ? so run_infer must fail FAST via the status probe instead of spinning the
poll budget."""
122
123         EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-
crash"
124
125         def __init__(self, *, state: str = "Failed"):
126             self.calls: list[dict] = []
127             self.cancelled: list[str] = []
128             self.status_polls = 0
129             self._state = state
130
131         def run_job(self, job_name, argv, *, region, project=None):
132             self.calls.append(
133                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
134             )
135             return self.EXEC_NAME
136
137         def cancel_execution(self, execution):
138             self.cancelled.append(execution)
139
140         def execution_terminal_state(self, execution):
141             self.status_polls += 1
142             # Anti-hang guard: the fast path MUST abort on the first terminal observation.
If a
143             # regression kept polling despite a terminal status, trip here instead of
spinning.
144             assert self.status_polls <= 5, "status probed repeatedly ? fast path did not
abort the poll"
145             return self._state
146
147
148         # ----- factory (default-
OFF)
149
150
151         def _cfg(compute_target="", storage_backend="local"):
152             return AppConfig.model_validate(
153                 {
154                     "deployment": {"compute_target": compute_target},
155                     "storage": {"backend": storage_backend},
156                 }
157             )
158
159
160         def test_factory_is_none_for_local_targets():

```

18. assistant (2026-07-07T15:51:12.361Z)

Let me verify the rebase integrity and confirm the commit is cleanly on top of master.

19. user (2026-07-07T15:51:16.667Z)

```

=== commits unique to HEAD vs origin/master ===
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
=== merge-base ===
20862d79cbe92ba1939ee964c4a57ab391f20ff7
=== is origin/master an ancestor of HEAD? ===
NO: diverged

```



```
=== files touched by HEAD commit ===
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
backend/compute/__init__.py | 8 +-
backend/compute/base.py | 14 +++
backend/compute/cloud_run.py | 115 ++++++-----
backend/worker/__main__.py | 16 +-
tests/test_compute_backend.py | 229 ++++++-----
5 files changed, 375 insertions(+), 7 deletions(-)
```

20. user (2026-07-07T15:51:32.755Z)

```
origin/master = ec2d080f3c63738de507bce0099b7cd42e2d397a
HEAD~1 = 20862d79cbe92ba1939ee964c4a57ab391f20ff7
=== commits on origin/master NOT in HEAD ===
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
=== (end) ===
```

21. user (2026-07-07T15:51:50.338Z)

```
=== #511 (ec2d080) files ===
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
backend/orchestration.py | 49 ++++++
tests/test_api_cloud_dispatch.py | 179 ++++++
2 files changed, 228 insertions(+)
=== does #511 touch any of my 5 files? ===
backend/orchestration.py
```

22. assistant (2026-07-07T15:52:20.168Z)

Let me check for any rebase duplication artifacts (duplicated field defs, duplicated flag appends, conflict markers).

23. user (2026-07-07T15:52:24.382Z)

```
=== conflict markers? ===
none
=== skip_validation occurrences ===
backend/compute/base.py:51: skip_validation: bool = False
backend/compute/cloud_run.py:252: if spec.skip_validation:
backend/worker/__main__.py:194: skip_validation=bool(getattr(args, "skip_validation",
False)),
backend/worker/__main__.py:282: if getattr(args, "skip_validation", False):
backend/orchestration.py:582: skip_validation=True,
=== --skip-validation appends ===
backend/compute/cloud_run.py:255: argv.append("--skip-validation")
backend/worker/__main__.py:754: "--skip-validation",
=== RemoteExecutionFailed occurrences ===
backend/compute/__init__.py:20: RemoteExecutionFailed,
backend/compute/__init__.py:28: "RemoteExecutionFailed",
backend/compute/base.py:20:class RemoteExecutionFailed(RuntimeError):
backend/compute/cloud_run.py:17: (`RemoteExecutionFailed`) instead of spinning the full
~65-min budget on a futile wait.
backend/compute/cloud_run.py:43: RemoteExecutionFailed,
backend/compute/cloud_run.py:280: # RemoteExecutionFailed (the fast negative path in
_poll_check) is NOT a PolicyTimeout,
backend/compute/cloud_run.py:321: ``RemoteExecutionFailed`` (abort the poll immediately ?
the execution is over, no marker
backend/compute/cloud_run.py:336: raise RemoteExecutionFailed(
backend/worker/__main__.py:183: * rc 5 ? the execution has TERMINATED without a marker
(`RemoteExecutionFailed`): a worker
backend/worker/__main__.py:186: from backend.compute import RemoteExecutionFailed
backend/worker/__main__.py:198: except RemoteExecutionFailed as e:
```

24. assistant (2026-07-07T15:53:06.893Z)

All feature wiring is clean and singular. Let me run the targeted tests for this change to confirm the integration behaves (both the fast-fail and the preserved skip_validation paths).

25. user (2026-07-07T15:53:14.338Z)

```
..... [100%]  
10 passed, 21 deselected in 2.18s
```

26. assistant (2026-07-07T15:53:17.938Z)

Let me run the full compute-backend test file to confirm no sibling regressions.

27. user (2026-07-07T15:53:29.167Z)

```
..... [100%]  
31 passed in 6.58s
```

28. assistant (2026-07-07T15:53:58.241Z)

The rebase is clean and both sibling features plus the new fix are intact and orthogonal. All 31 compute-backend tests pass, including the 10 new fast-fail/skip_validation integration tests. No defect-grade findings.

Key verifications:

- Rebase integrity: HEAD is a single commit atop 20862d7, whose tree already contains both #513 (skip_validation) and #512 (validators). The three-dot diff shows only the intended 5-file change; no conflict markers; no duplicated field/flag/class definitions.
- (a) `ComputeJobSpec.skip\_validation` field survives (base.py:51) and `run\_infer` still appends `--skip-validation` when set (cloud_run.py:252-255).
- (b) Both callers of `run\_infer` handle the new mode. worker `\_dispatch\_remote` builds the spec with `skip\_validation` (worker/`\_\_main\_\_`.py:194) and catches `RemoteExecutionFailed`?rc 5 (198-202) alongside `PolicyTimeout`?rc 4 (203-207); the two exception types are unrelated (RuntimeError vs TimeoutError) so both arms are reachable. The actual control-plane path for the motivating scenario ? orchestration `\_maybe\_dispatch\_remote` ? builds the spec with `skip\_validation=True` (orchestration.py:582) and its broad `except Exception` (593) catches `RemoteExecutionFailed`, mapping it to a clean `\_failed(...)` response (no unhandled 500). The old-image `--skip-validation`?argparse-exit-2?no-marker case therefore fails fast in the control plane where the new probe actually runs.
- (c) Orthogonal, no shadowing: skip_validation is argv construction at dispatch; the fast-fail is a poll-time Executions-API probe. `poll\_until` does not swallow `check()` exceptions, so a raised `RemoteExecutionFailed` propagates past the `except PolicyTimeout` to the caller. The marker stays the sole success signal (checked first; an empty-but-present marker still resolves), the probe only fires when the marker object is absent, and the one post-terminal race re-check plus GCS read-after-write consistency (marker is pushed before container exit) rule out a false-positive on a healthy run. Cloud Run task retries keep `completion\_time` unset so the probe returns None and polling continues until retries are exhausted ? no premature failure.

One non-defect note: origin/master has since advanced to ec2d080 (#511), which this branch does not yet include, but #511 touches only orchestration.py ? a file this commit does not modify ? so re-rebasing onto the current tip will not conflict on any of the 5 changed files.

29. user (2026-07-07T15:54:09.391Z)

Structured output provided successfully